Parallel and Distributed Computing Overview

(CS 3006)

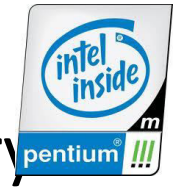
Muhammad Aadil Ur Rehman

Department of Computer Science,
National University of Computer & Emerging Sciences,
Islamabad Campus

Credits: Dr. Muhammad Aleem, Dr. Muhammad Arshad Islam

Single Processor Systems

– From **mid-1980**s until **2004 computers got faster**:



- **More** number of **transistors** were **packed** every year.
- **Size** of **transistors** continued to get **smaller**

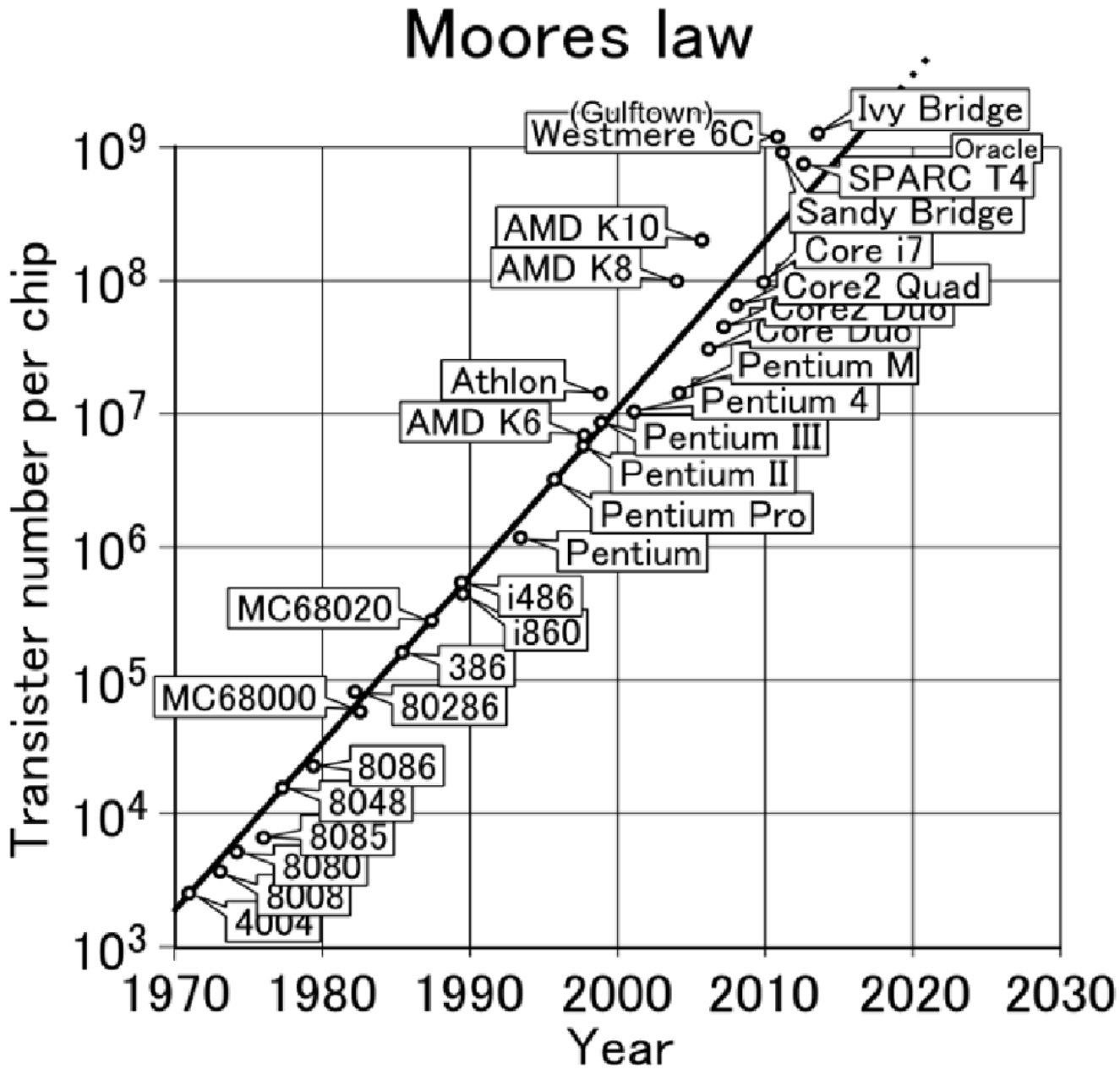
- **Resulted** in **faster processors** (**more GHzs** means **more performance**) - **as observed in Moore's law**
- frequency scaling →
  CPU Time = IC × CPI x Clock Cycle Time
  Clock Cycle Time = 1 / Clock Rate

Moore's Law

- **An observation by Gordon Moore** (co-founder of Intel)
- **Projection** based on historical trend (**processor design**) in 1970s

Number of transistors in a processor doubles every year

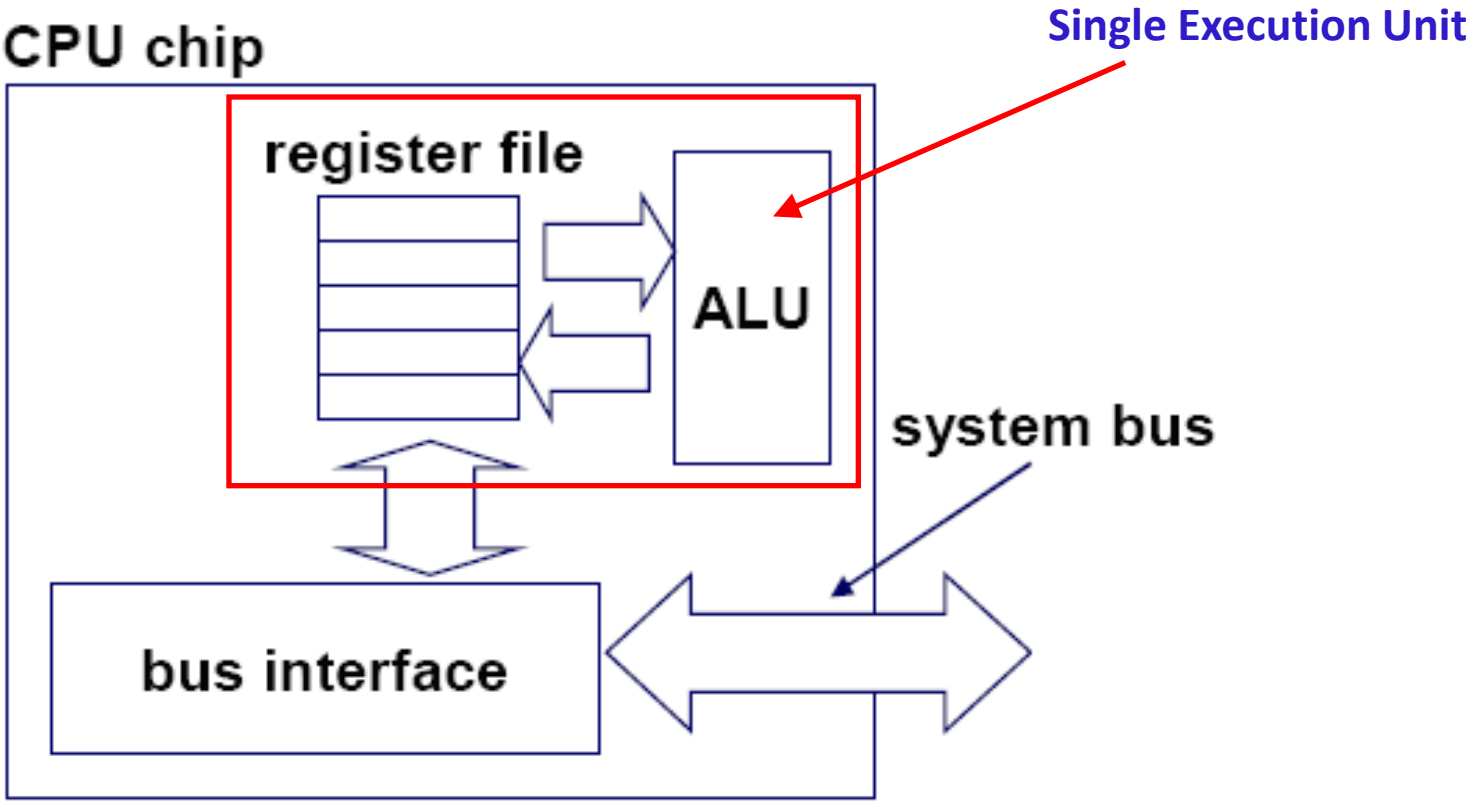
- The **real-data shows** the **correlation**:
  - ***Number of transistors doubled every 18 months***



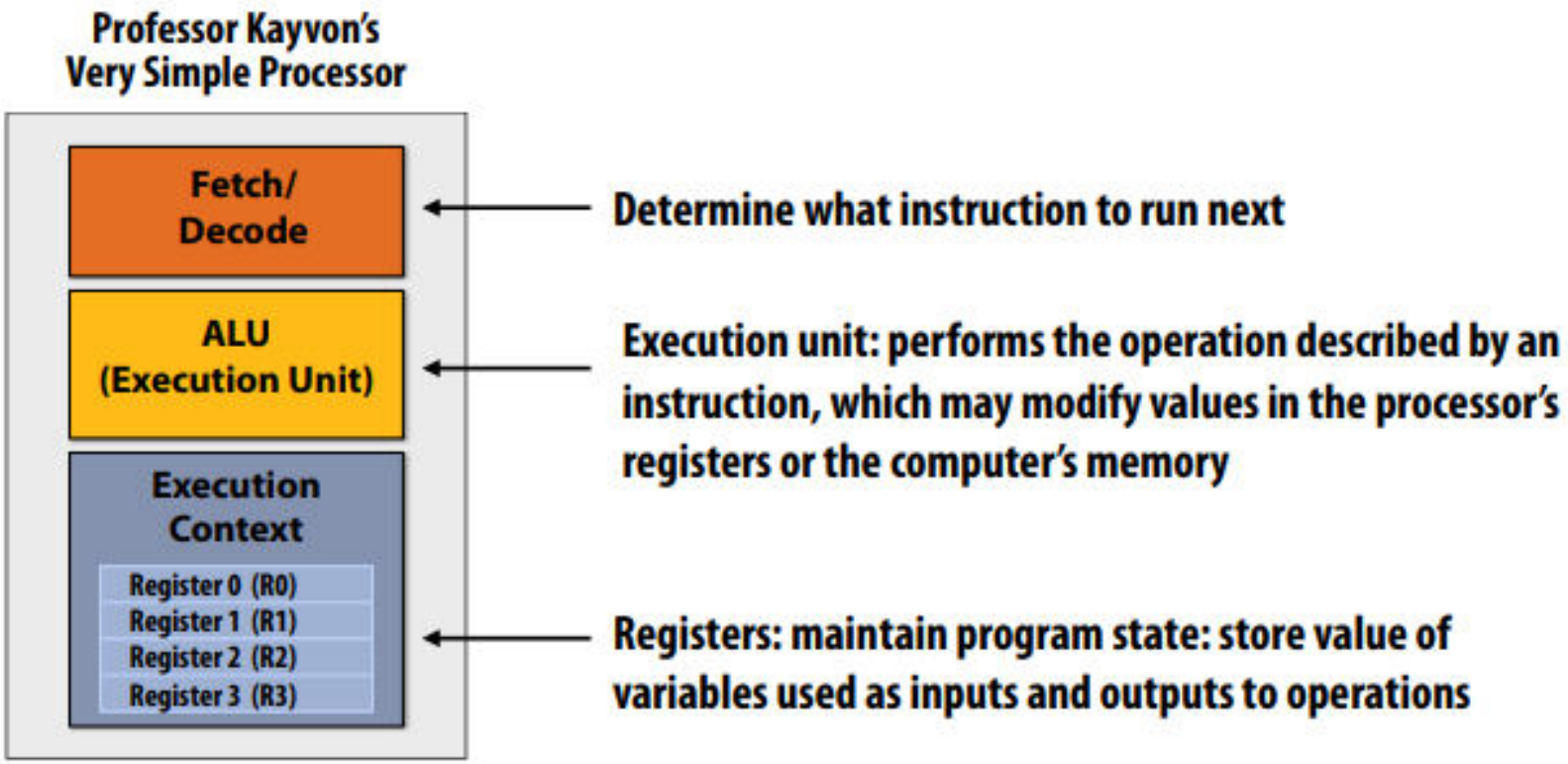
ACK: shigeru23 CC BY-SA 3.0, via Wikimedia Commons

What does a processor do?

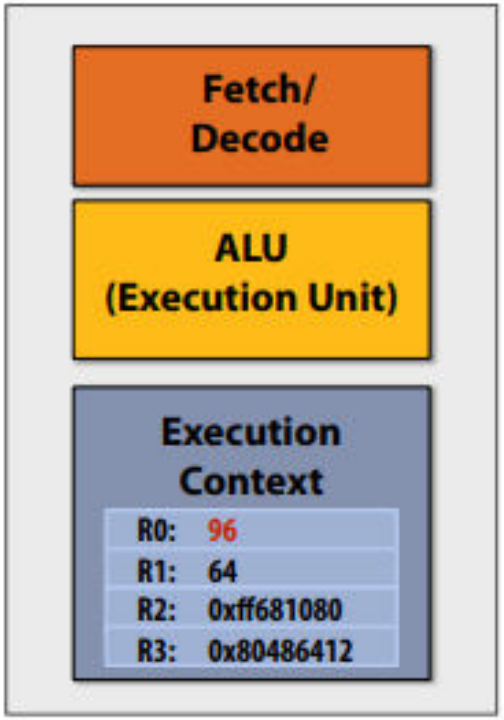
Typical Single-core Processor



A processor executes instructions



One example instruction: add two numbers



Step 1:
Processor gets next program instruction from memory (figure out what the processor should do next)

`add R0 ← R0, R1`

"Please add the contents of register R0 to the contents of register R1 and put the result of the addition into register R0"

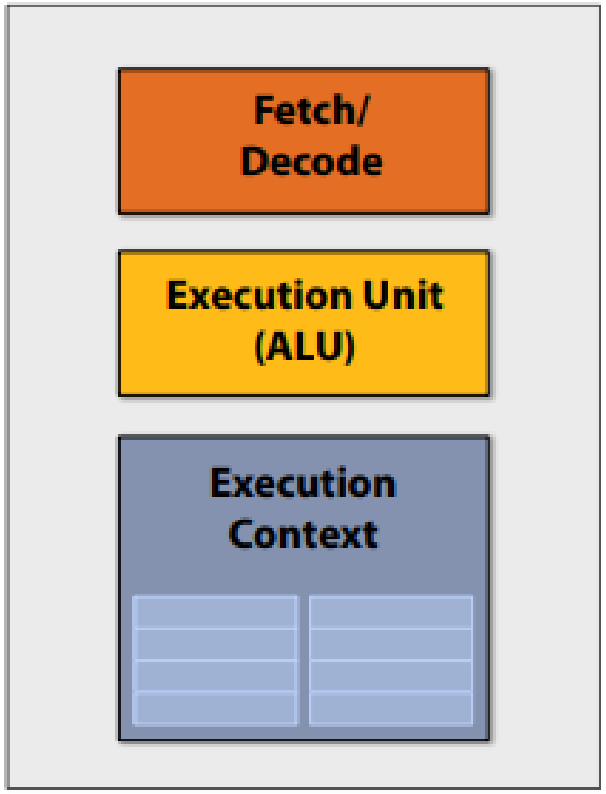
Step 2:
Get operation inputs from registers
Contents of R0 input to execution unit: 32
Contents of R1 input to execution unit: 64

Step 3:
Perform addition operation:
Execution unit performs arithmetic, the result is: 96

Step 4:
Store result 96 **back to register R0**

Execute program

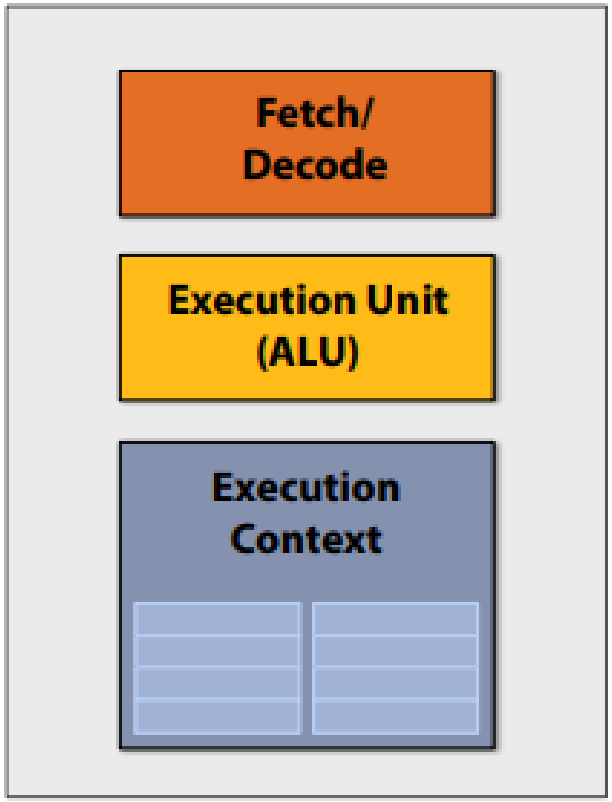
My very simple processor: executes one instruction per clock




```
ld   r0, addr[r1]
mul  r1, r0, r0
mul  r1, r1, r0
...
...
...
...
...
...
st   addr[r2], r0
```

Execute program

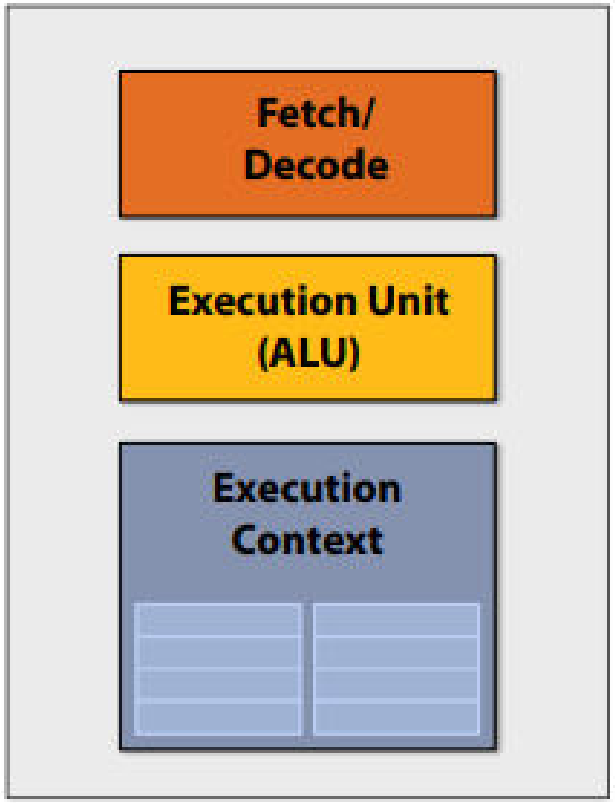
My very simple processor: executes one instruction per clock




```
ld   r0, addr[r1]
mul  r1, r0, r0
mul  r1, r1, r0
...
...
...
...
...
...
st   addr[r2], r0
```

Execute program

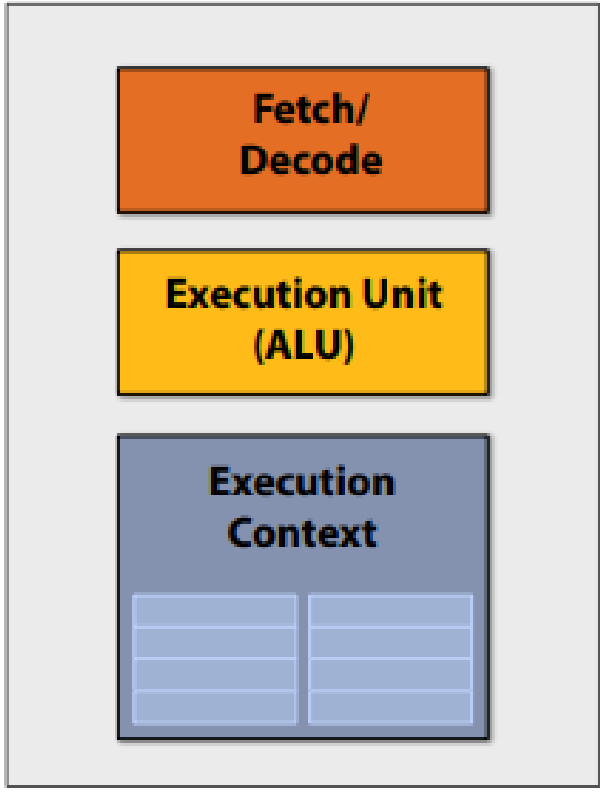
My very simple processor: executes one instruction per clock




```
ld   r0, addr[r1]
mul  r1, r0, r0
mul  r1, r1, r0
...
...
...
...
...
...
st   addr[r2], r0
```

Execute program

My very simple processor: executes one instruction per clock




```
ld   r0, addr[r1]
mul  r1, r0, r0
mul  r1, r1, r0
...
...
...
...
...
...
st   addr[r2], r0
```

Lets consider a very simple piece of code

```
a = x*x + y*y + z*z
```

Consider the following five instruction program:

Assume register R0 = x, R1 = y, R2 = z

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

R3 now stores value of program variable 'a'

This program has five instructions, so it will take five clocks to execute, correct?

Can we do better?

What if up to two instructions can be performed at once?

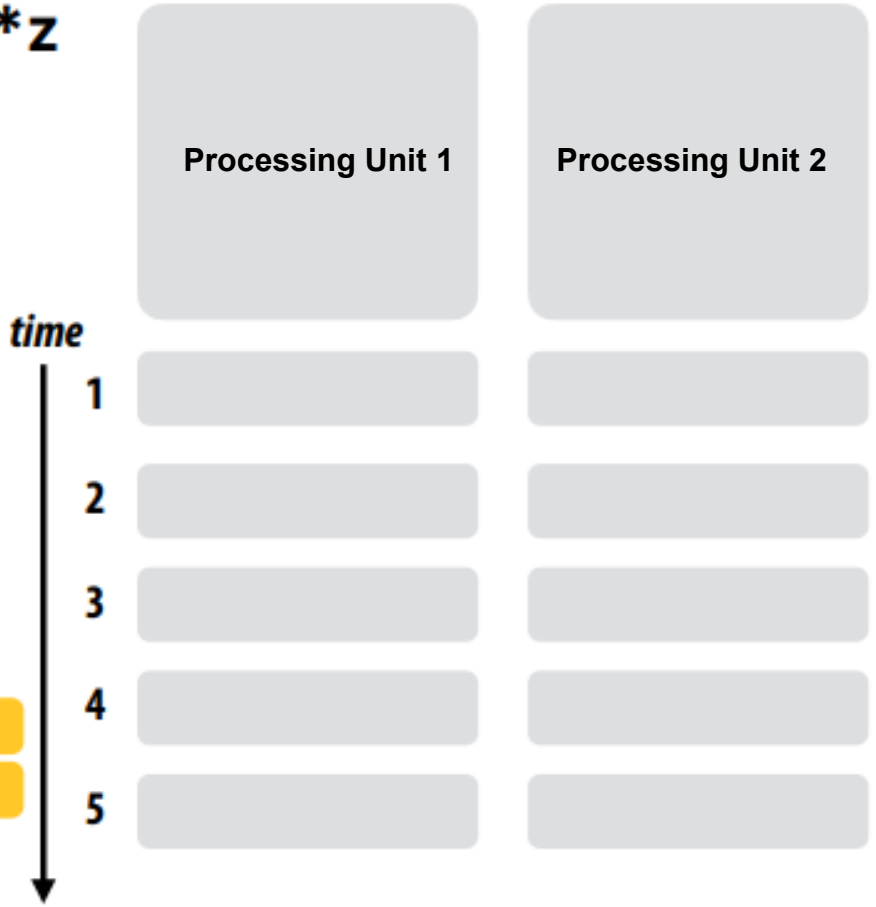
```
a = x*x + y*y + z*z
```

Assume register
R0 = x, R1 = y, R2 = z

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

R3 now stores value of program variable 'a'

1. `mul R0, R0, R0`
2. `mul R1, R1, R1`
3. `mul R2, R2, R2`
4. `add R0, R0, R1`
5. `add R3, R0, R2`



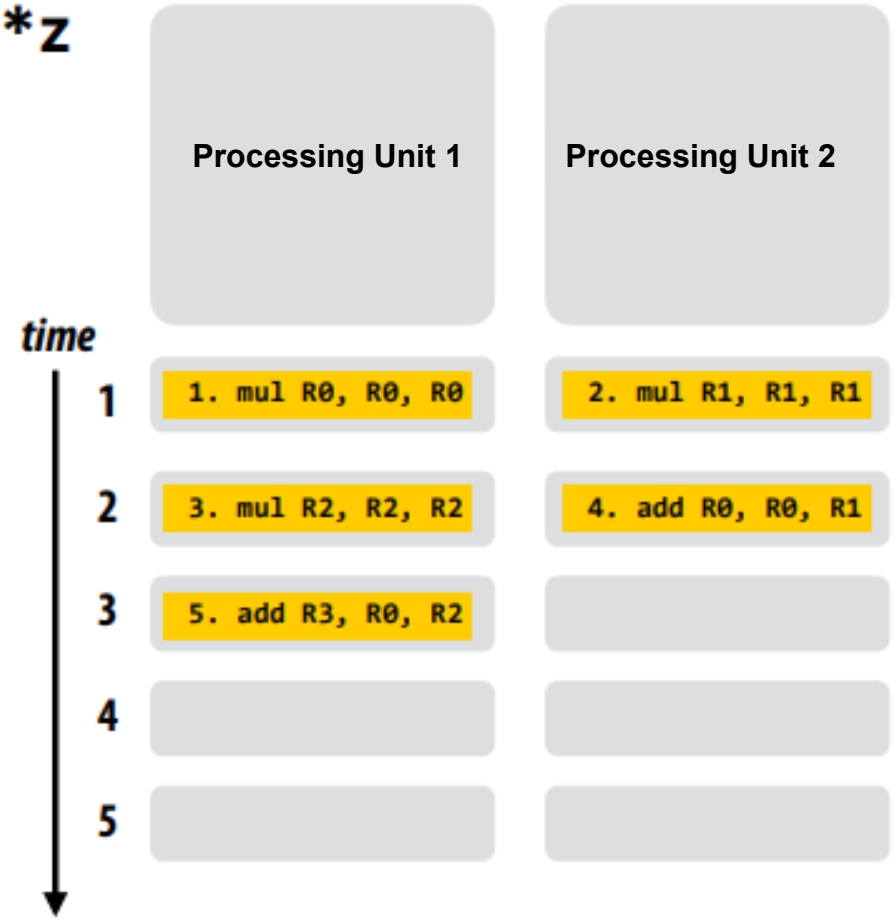
What if up to two instructions can be performed at once?

a = x*x + y*y + z*z

Assume register
R0 = x, R1 = y, R2 = z

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

R3 now stores value of program variable 'a'



Program Order

What does it mean for our parallel to scheduling to that "respects program order"?

Hint: What is expected of the output.

Program Order

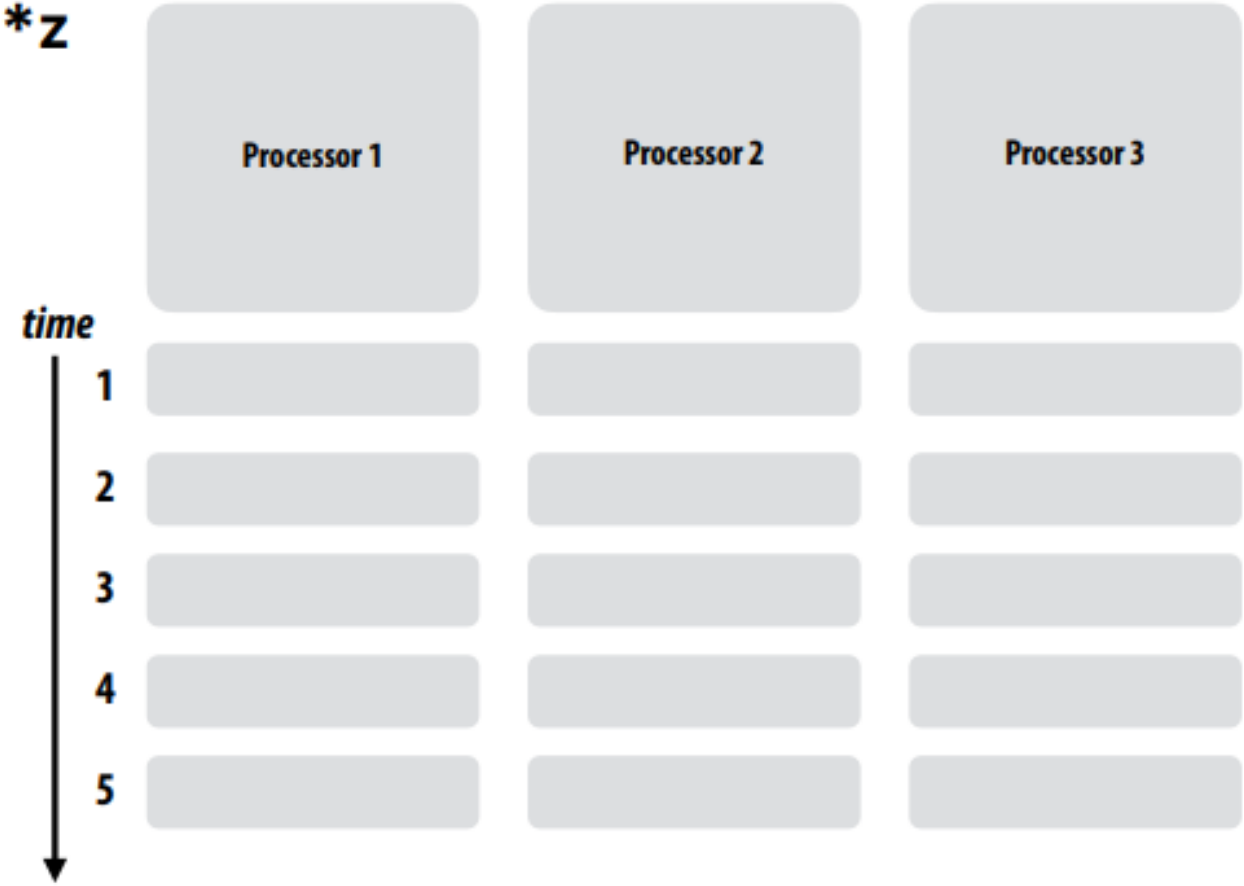
What about three instructions at once?

```
a = x*x + y*y + z*z
```

Assume register
R0 = x, R1 = y, R2 = z

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

R3 now stores value of program variable 'a'



Program Order

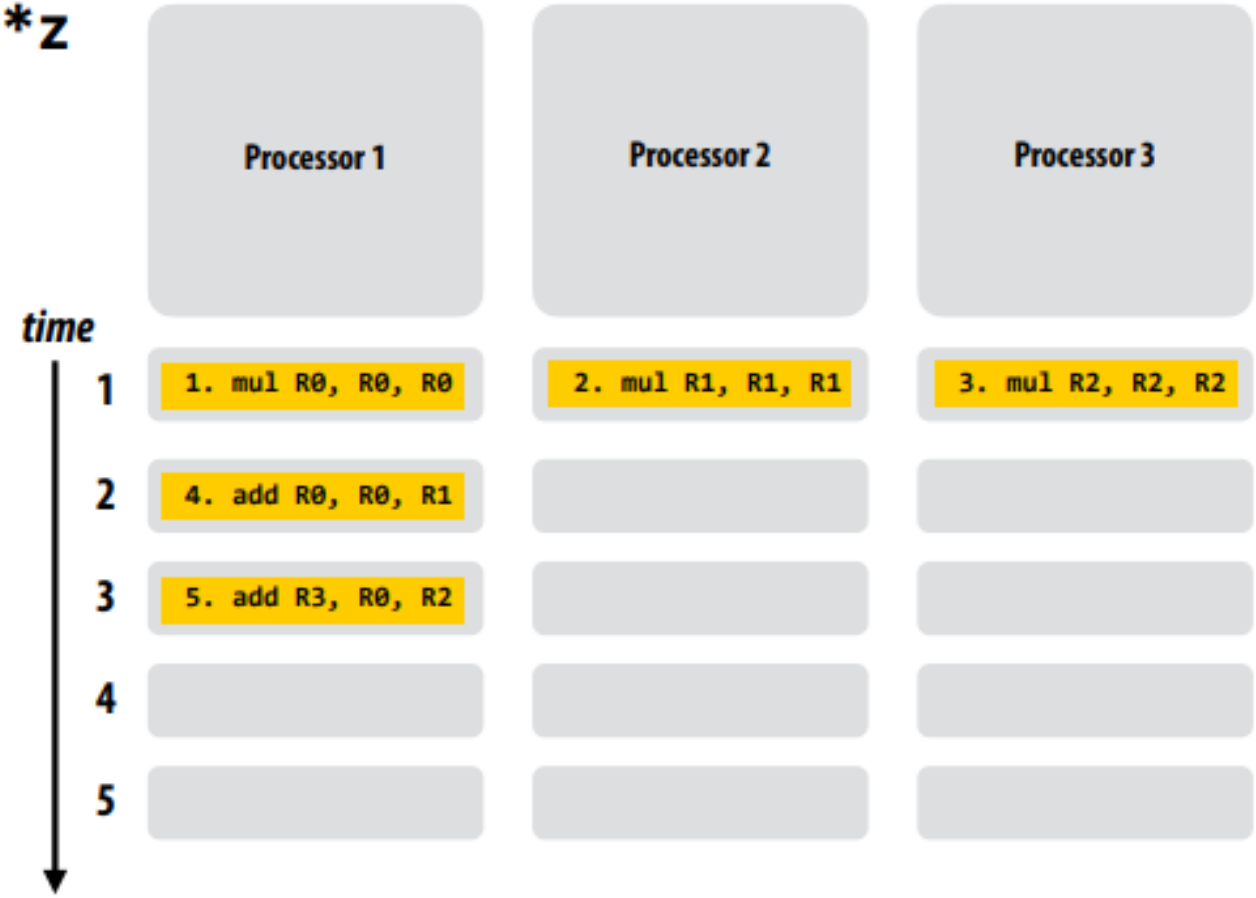
What about three instructions at once?

```
a = x*x + y*y + z*z
```

Assume register
R0 = x, R1 = y, R2 = z

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

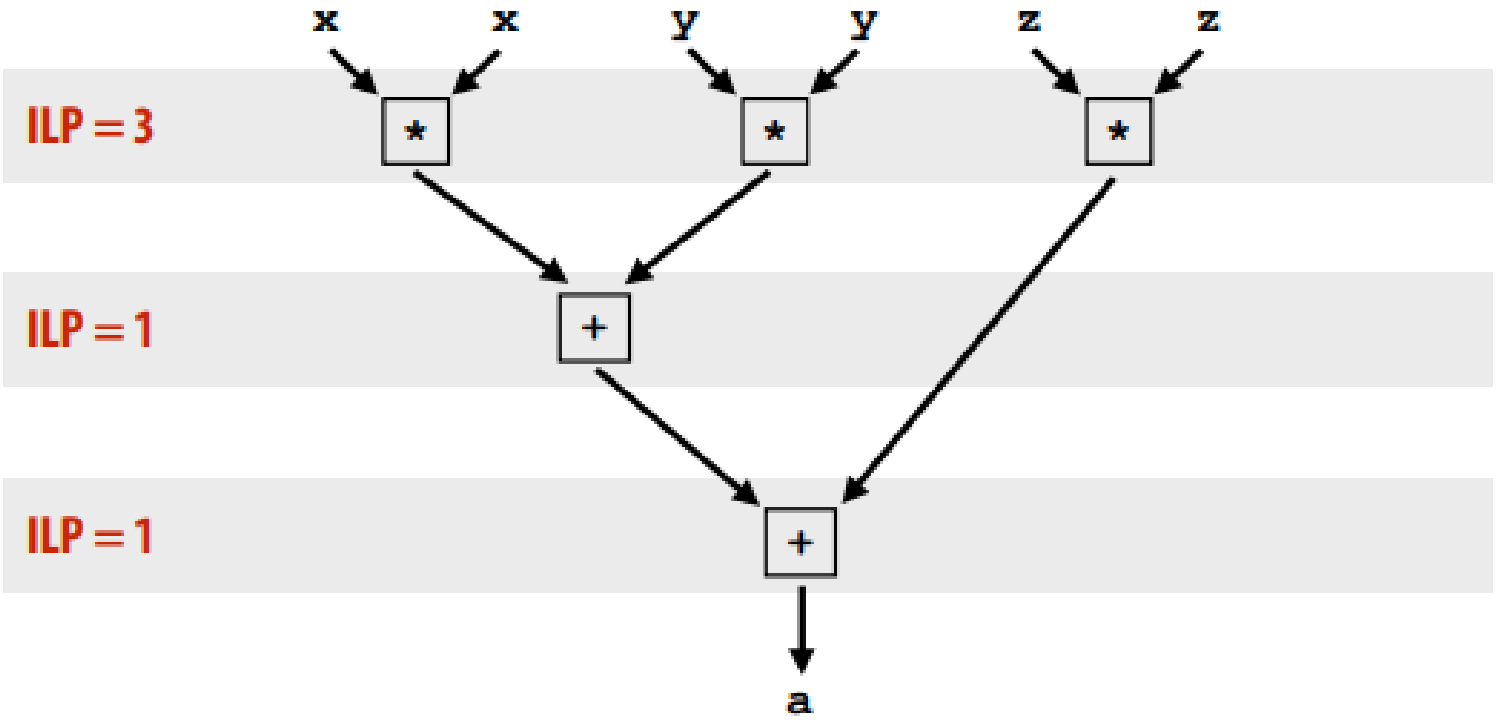
R3 now stores value of program variable 'a'



Instruction level parallelism (ILP)

- ILP = 3

```
a = x*x + y*y + z*z
```



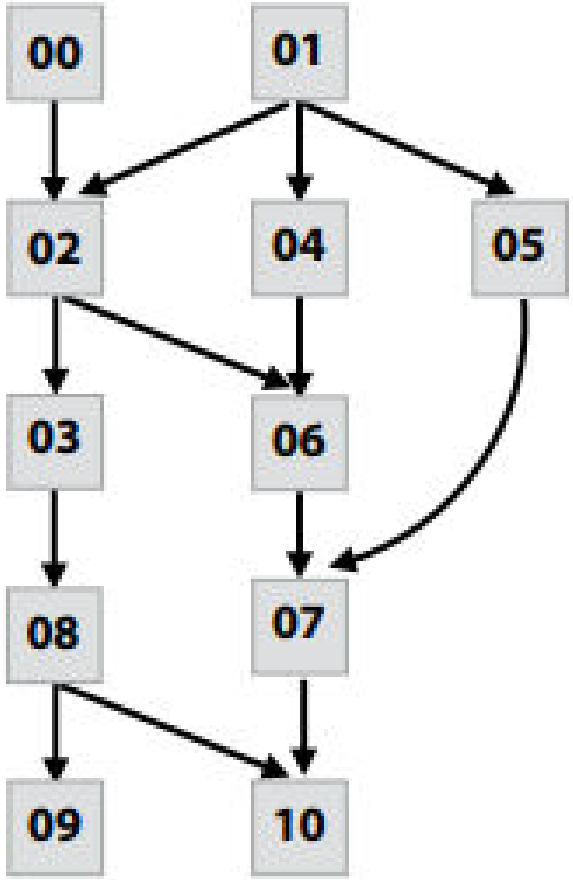
Instruction Dependency Graph

A more complex example

Program (sequence of instructions)

PC	Instruction	Computed value
00	a = 2	
01	b = 4	
02	tmp2 = a + b	// 6
03	tmp3 = tmp2 + a	// 8
04	tmp4 = b + b	// 8
05	tmp5 = b * b	// 16
06	tmp6 = tmp2 + tmp4	// 14
07	tmp7 = tmp5 + tmp6	// 30
08	if (tmp3 > 7)	
09	print tmp3	
	else	
10	print tmp7	

Instruction dependency graph



Superscalar processor execution

```
a = x*x + y*y + z*z
```

Assume register
R0 = x, R1 = y, R2 = z

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

Idea #1:

Superscalar execution: processor automatically finds* independent instructions in an instruction sequence and executes them in parallel on multiple execution units!

In this example: instructions 1, 2, and 3 can be executed in parallel without impacting program correctness (on a superscalar processor that determines that the lack of dependencies exists)

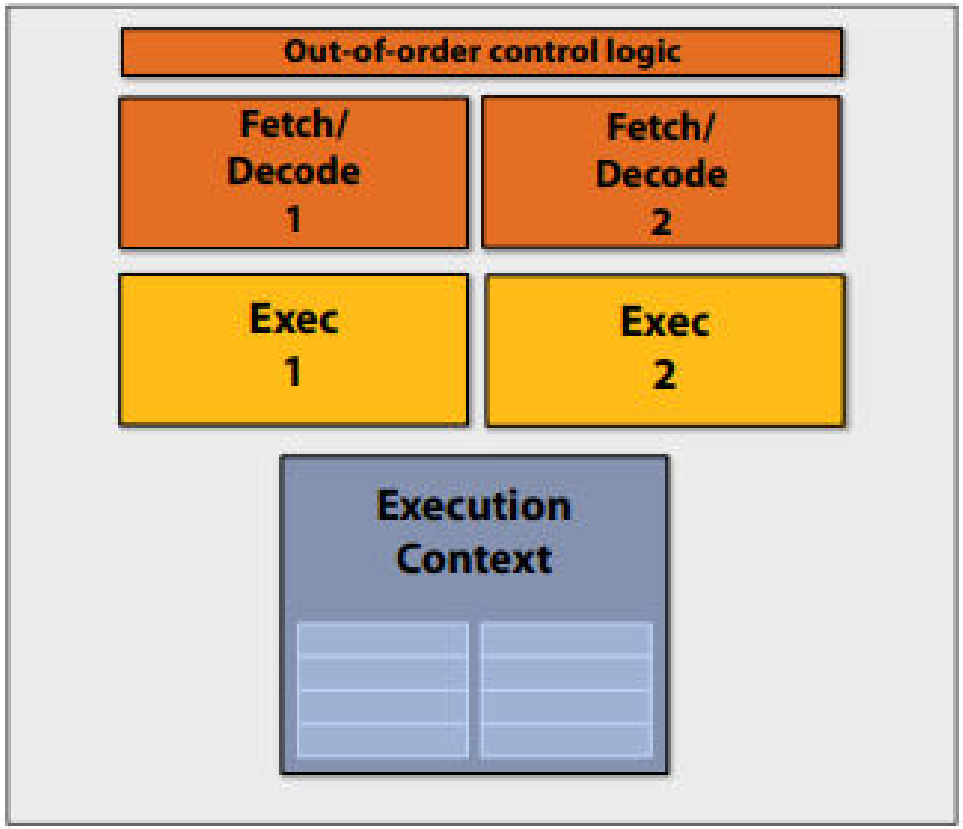
But instruction 4 must be executed after instructions 1 and 2

And instruction 5 must be executed after instruction 4

*** Or the compiler finds independent instructions at compile time and explicitly encodes dependencies in the compiled binary.**

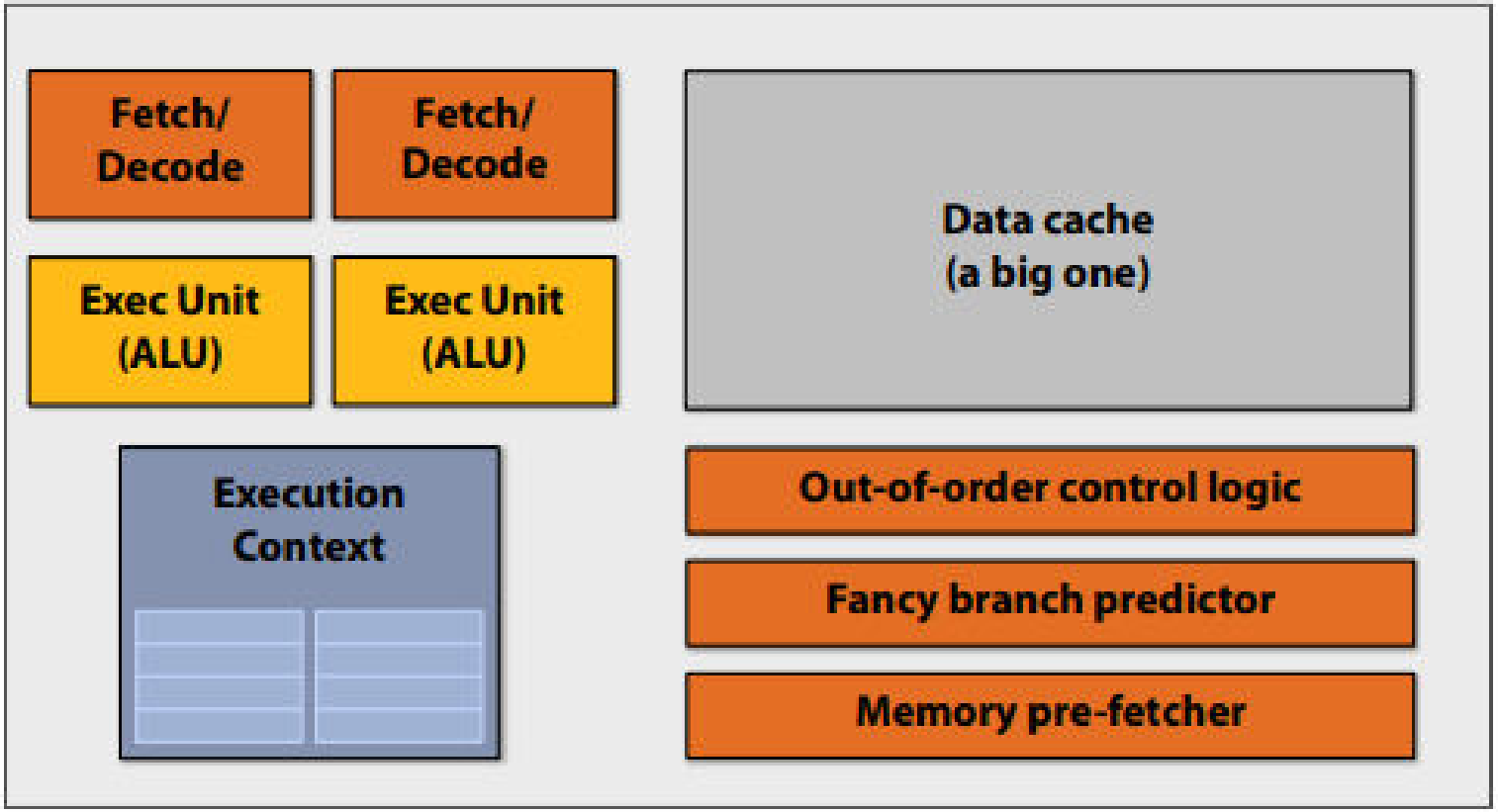
Superscalar processor

This processor can decode and execute up to two instructions per clock



Pre multi-core era processor

Majority of chip transistors used to perform operations that help make a single instruction stream run fast



More transistors = larger cache, smarter out-of-order logic, smarter branch predictor, etc.

Old Intel Pentium 4 CPU

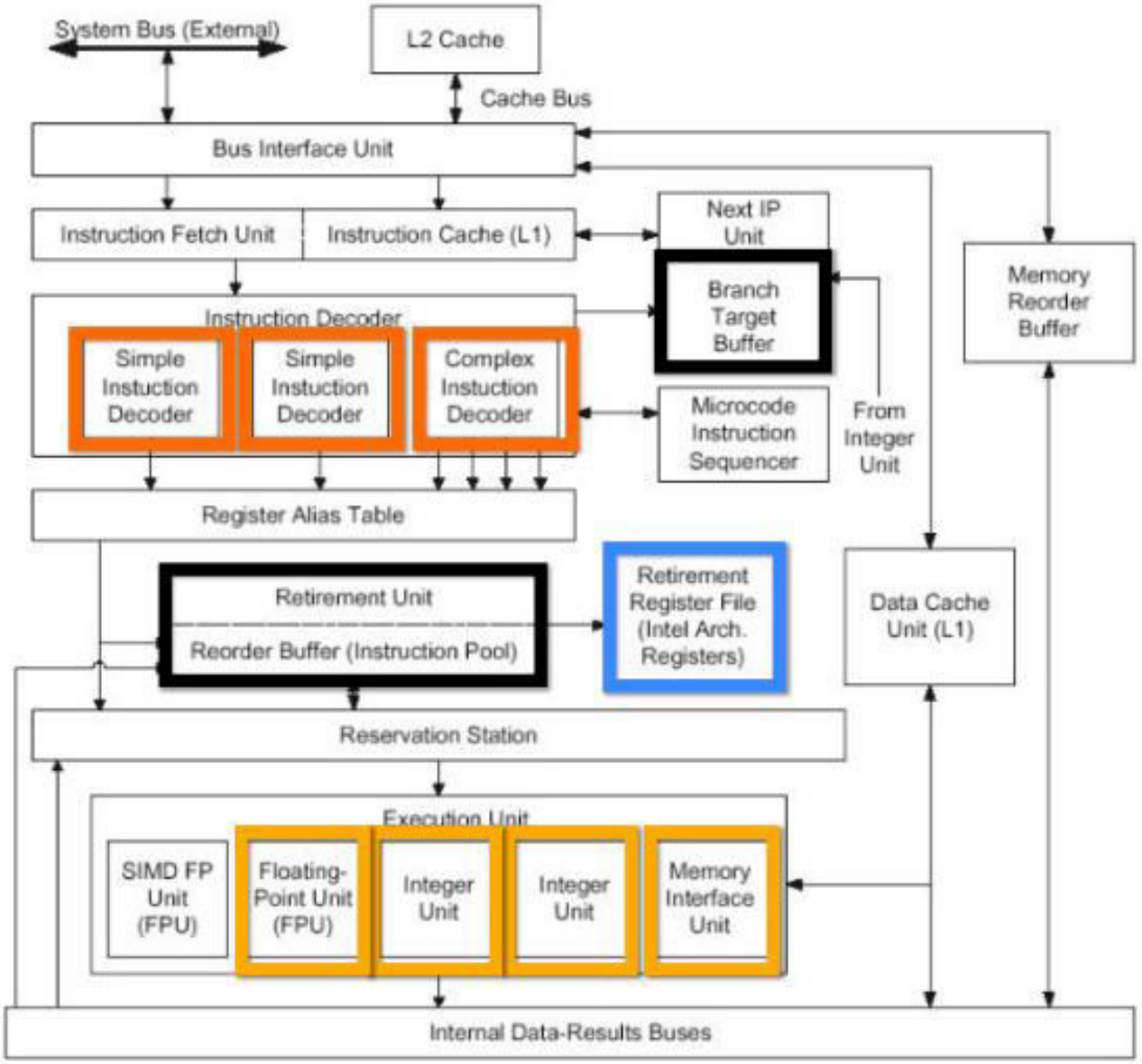


Image credit: http://ixbtlabs.com/articles/pentium4/index.html

Another Example

```
void sinx(int N, int terms, float* x, float* y)
{
   for (int i=0; i<N; i++)
   {
      float value = x[i];
      float numer = x[i] * x[i] * x[i];
      int denom = 6;  // 3!
      int sign = -1;

      for (int j=1; j<=terms; j++)
      {
         value += sign * numer / denom;
         numer *= x[i] * x[i];
         denom *= (2*j+2) * (2*j+3);
         sign *= -1;
      }

      y[i] = value;
   }
}
```

Compute $\sin(x)$ using Taylor expansion:

$$\sin(x) = x - x^3/3! + x^5/5! - x^7/7! + \ldots$$

for each element of an array of N floating-point numbers

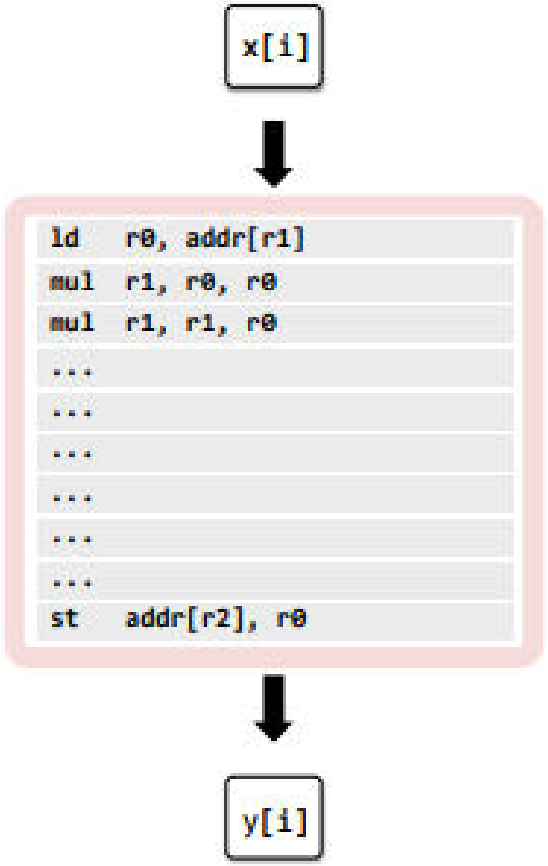
Another Example

Compile program

```
void sinx(int N, int terms, float* x, float* y)
{
   for (int i=0; i<N; i++)
   {
      float value = x[i];
      float numer = x[i] * x[i] * x[i];
      int denom = 6;  // 3!
      int sign = -1;

      for (int j=1; j<=terms; j++)
      {
         value += sign * numer / denom;
         numer *= x[i] * x[i];
         denom *= (2*j+2) * (2*j+3);
         sign *= -1;
      }

      y[i] = value;
   }
}
```

compiler

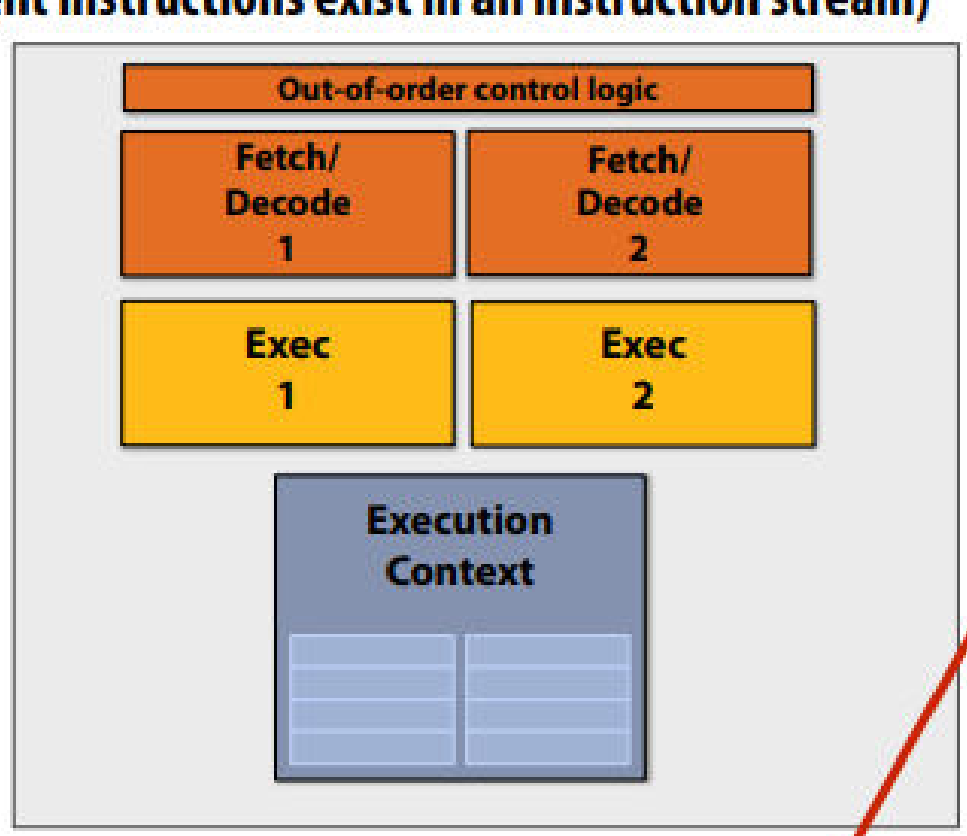
Compiled instruction stream (scalar instructions)



Another Example

Superscalar processor

The processor shown here can decode and execute two instructions per clock (if independent instructions exist in an instruction stream)

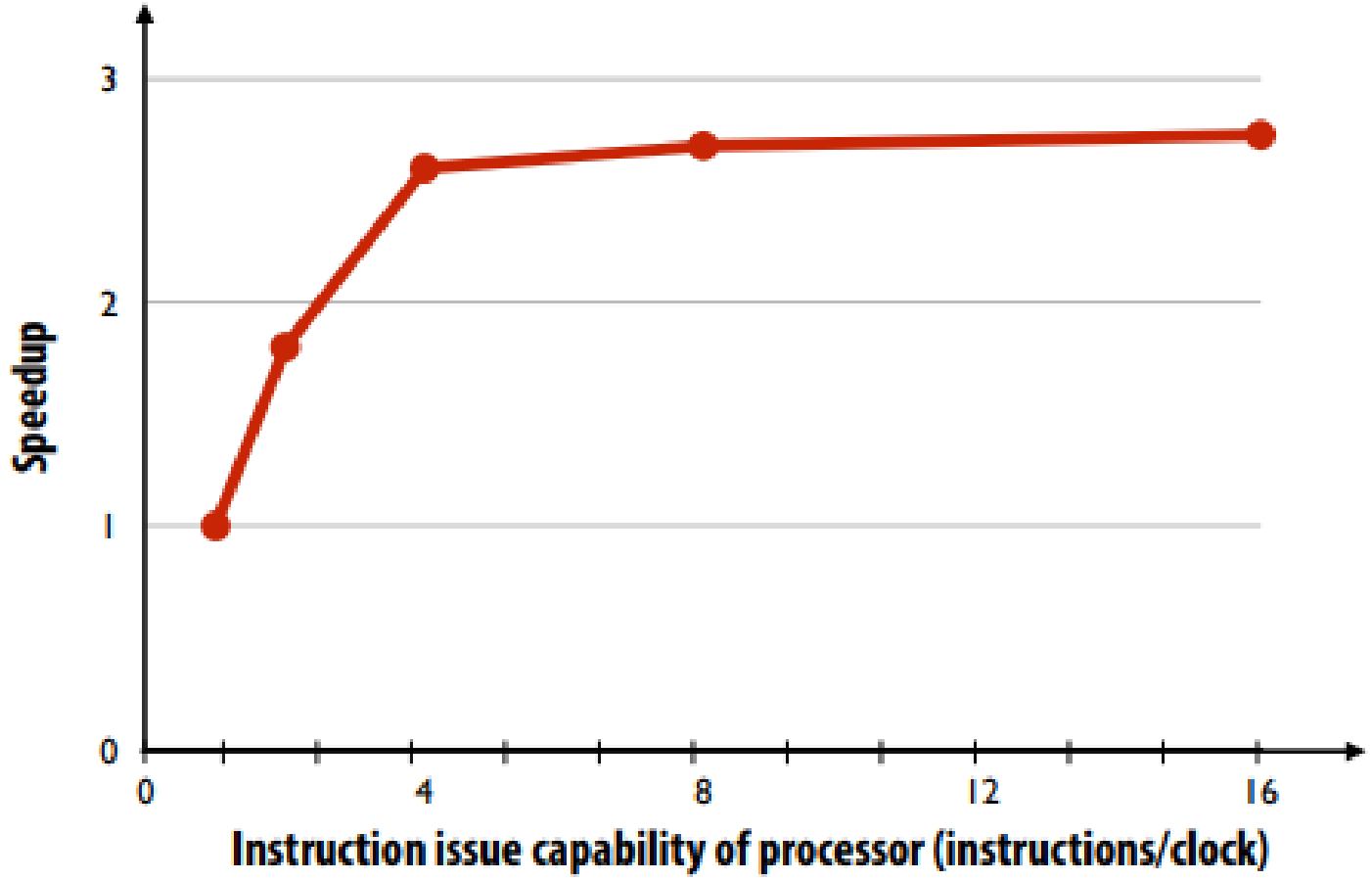



```
x[i]
ld   r0, addr[r1]
mul  r1, r0, r0
mul  r1, r1, r0
...
...
...
...
...
st   addr[r2], r0
y[i]
```

Note: No ILP exists in this region of the program

Diminishing returns of superscalar execution

Most available ILP is exploited by a processor capable of issuing four instructions per clock (Little performance benefit from building a processor that can issue more)



What went wrong with in building “A Faster Single Processor”?

Problems with single "Faster" CPU

- **More transistors --> more parallelism opportunities**
- **Implicit Parallelism** (hidden from programmer):
  - **Execution pipelines, multiple-functional units**
- **Explicit Parallelism:**
  - **VLIW, More execution Units**
- **Higher packing density** means **shorter** electrical **paths**, giving **higher performance**

Problems with single “Faster” CPU

– **Power Consumption:**

- With growing **clock-frequencies**, **power consumption** increases **exponentially,** hitting **Power Wall.**

- **Not possible forever**: a **higher frequency processor** could have **require** a **whole nuclear reactor** to fulfill the **energy needs**

https://www.nytimes.com/2004/05/17/business/technology-intel-s-big-shift-after-hitting-technical-wall.html

May 17, 2004 … Intel, the world's largest chip maker, publicly acknowledged that it had hit a "thermal wall" on its microprocessor line. As a result, the company is changing its product strategy and disbanding one of its most advanced design groups. Intel also said that it would abandon two advanced chip development projects …

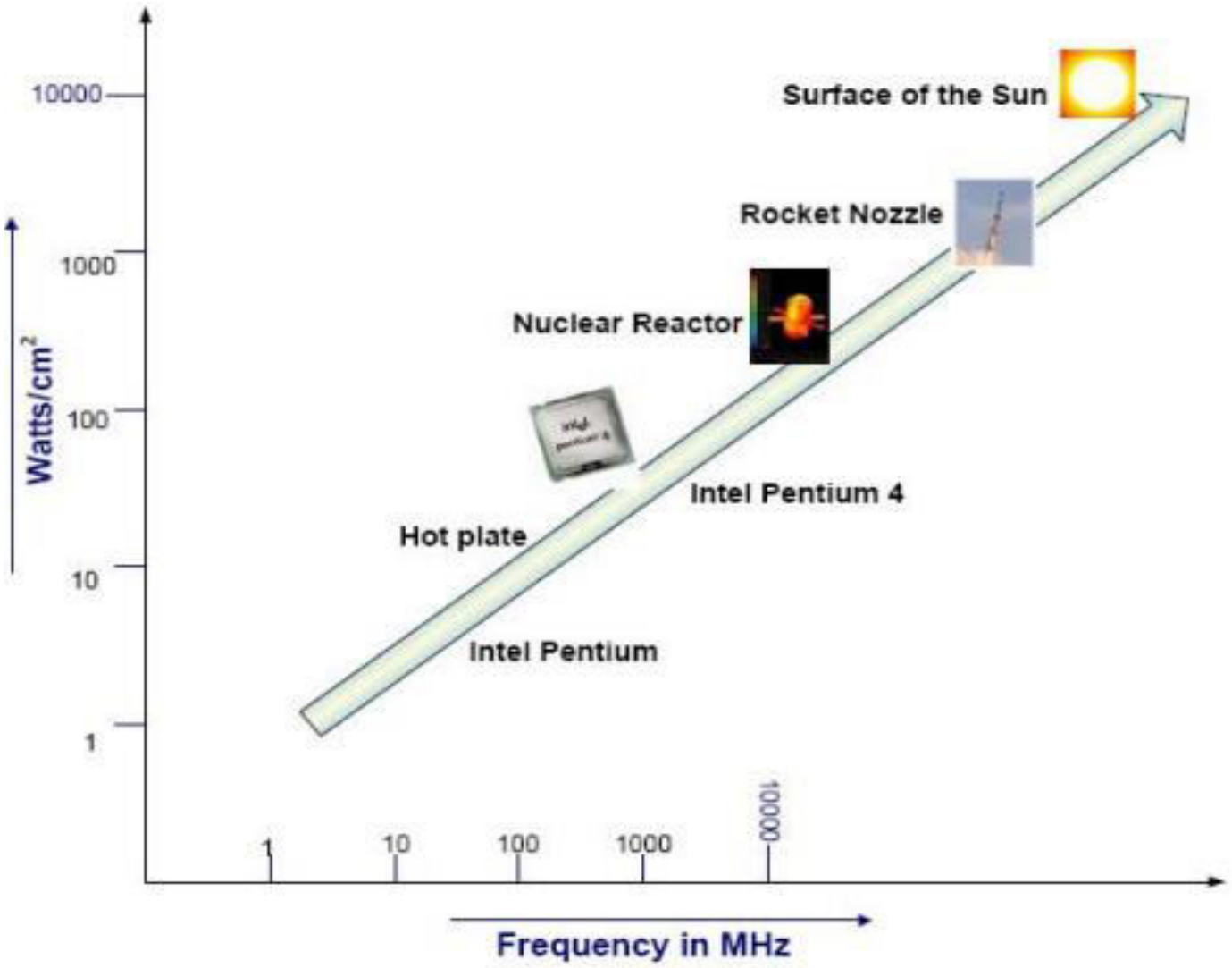
Now, Intel is embarked on a course already adopted by some of its major rivals: obtaining more computing power by stamping multiple processors on a single chip rather than straining to increase the speed of a single processor … Intel's decision to change course and embrace a "dual core" processor structure shows the challenge of overcoming the effects of heat generated by the constant on-off movement of tiny switches in modern computers … some analysts and former Intel designers said that *Intel was coming to terms with escalating heat problems so severe they threatened to cause its chips to fracture at extreme temperatures…*

Acknowledgements: Department of Computer Science Rice University

Problems with single “Faster” CPU

- **Heat Dissipation**:
  - With growing **clock frequency**, more **power** is **consumed** → **More heat** is **dissipated**.. **Thermal Wall**
  - **Keeping processors cool** become a **problem** (with increa uencies)

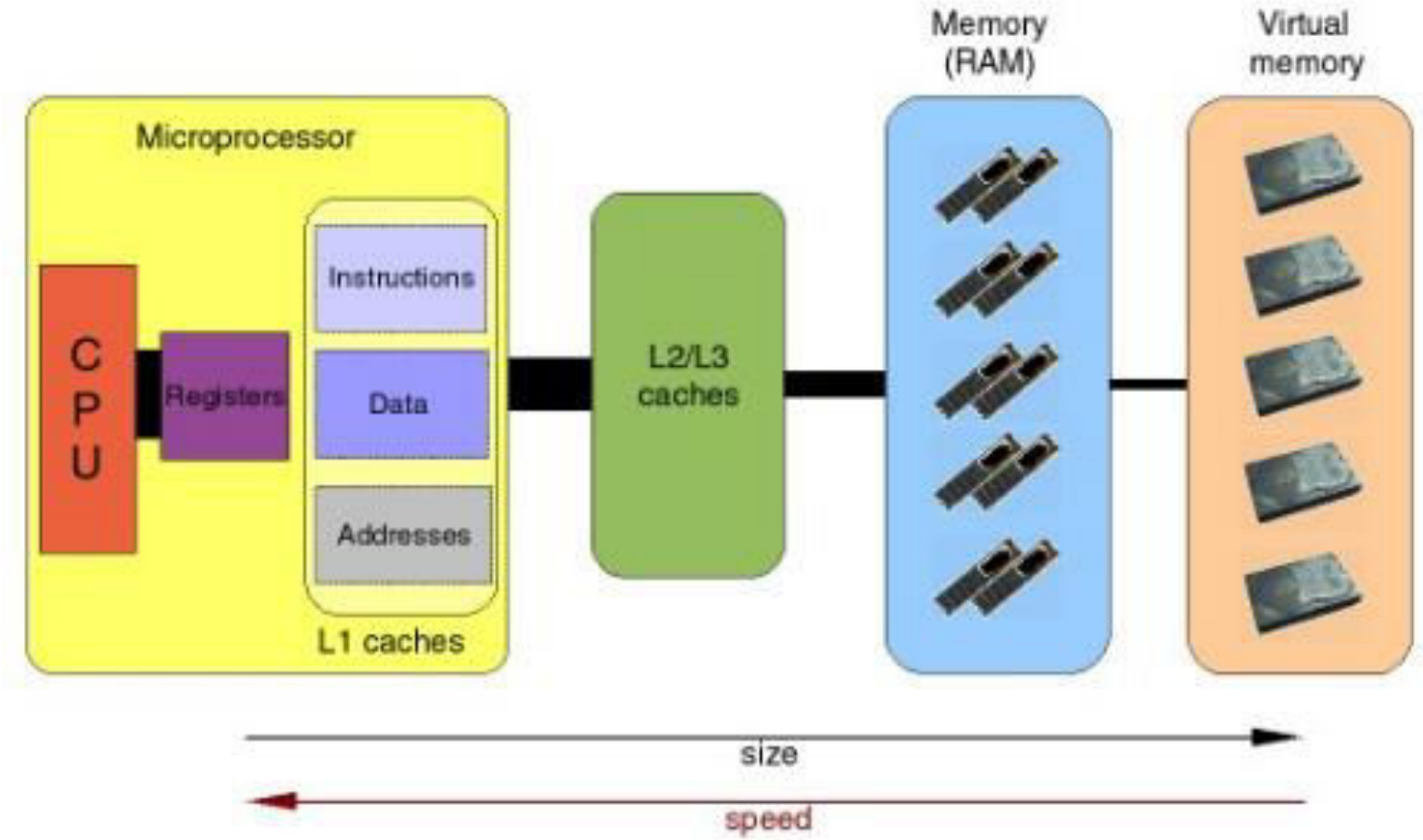
Problems with single “Faster” CPU



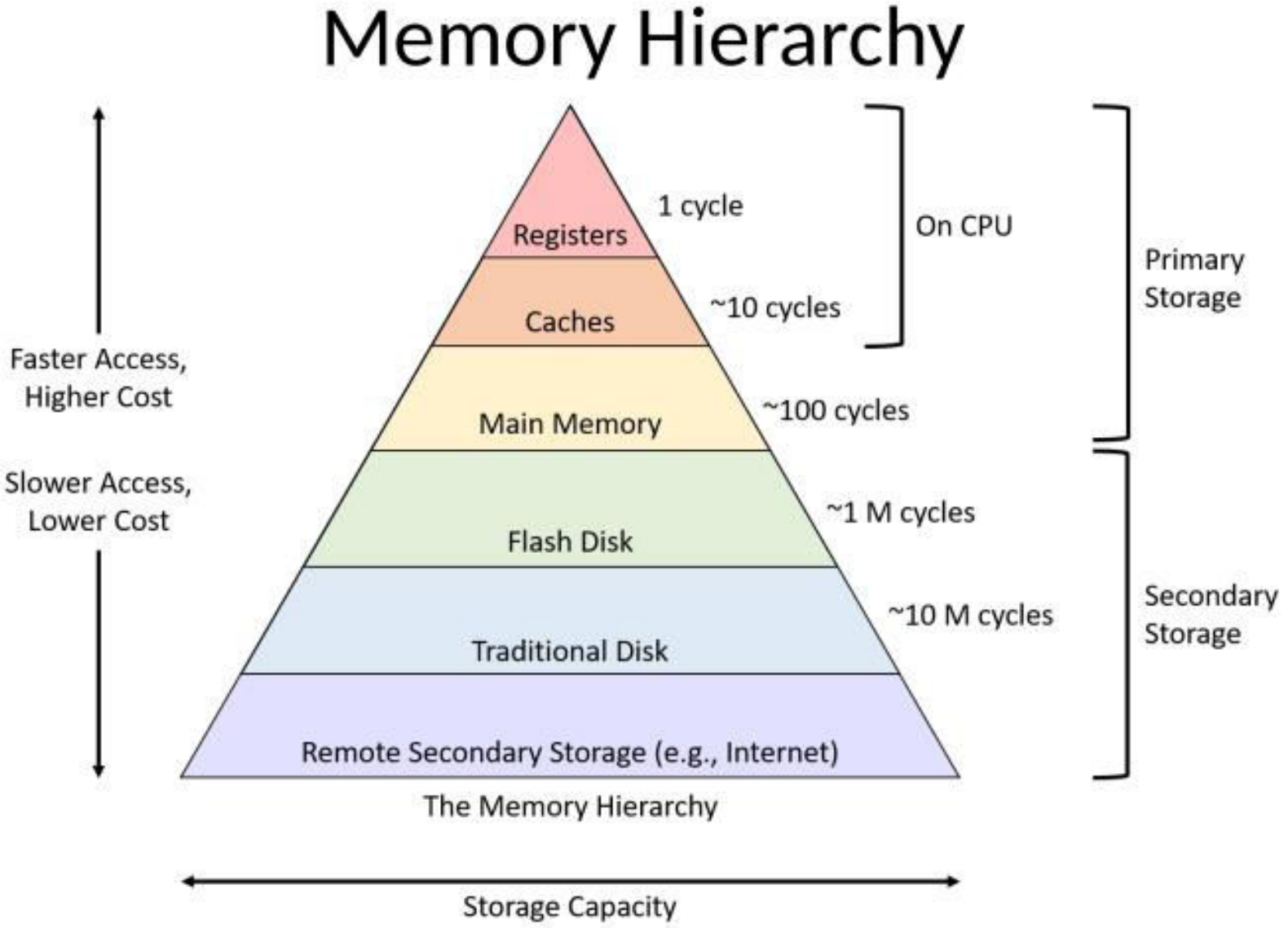
Other Issues…

- **Smaller size of transistors:**
  - **Fabrication** of **processors** become **more difficult**.
- **Limited memory size and Memory Wall:**
  - A **single processor** has **limited internal memory** i.e., cache memory L1, L2, registers, etc.
  - A **single processor** based **system** has **limited overall system memory** or main memory.
  - CPU speed increased at an average rate of 55% per year from 1986 to 2000, whereas **RAM speed increased** by just **10%** per year.

Single Processor Systems



Single Processor Systems



The Memory Hierarchy

ILP Tapped out + Power Wall + Memory Wall
** End of Frequency Scaling **

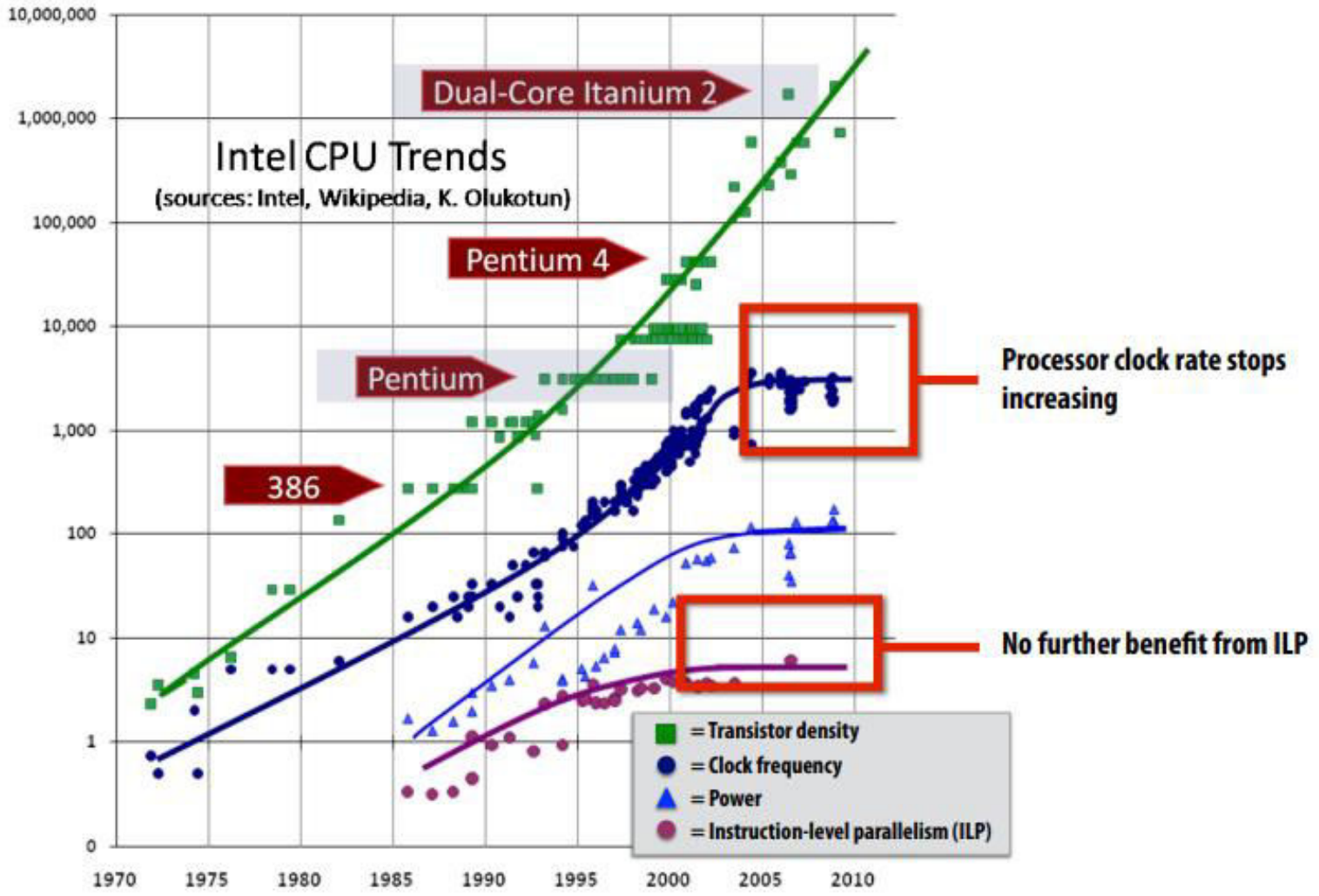
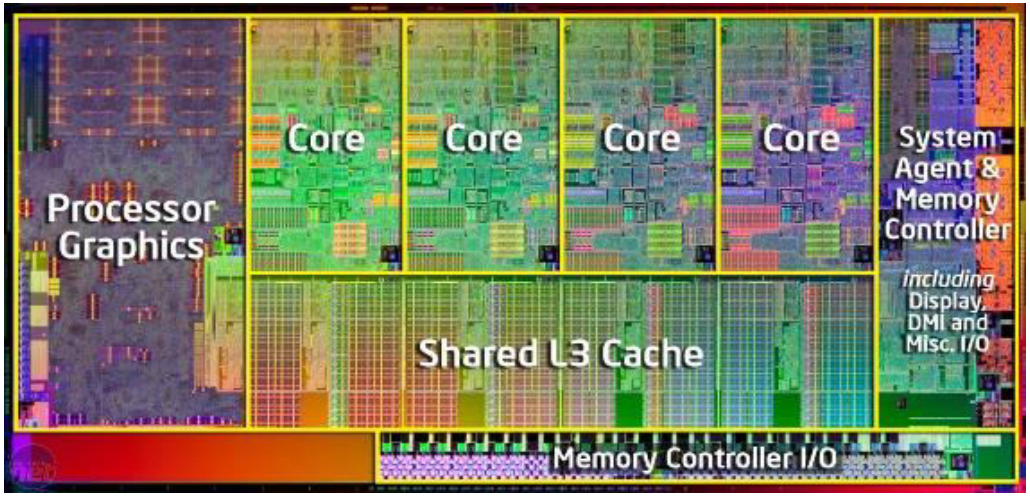


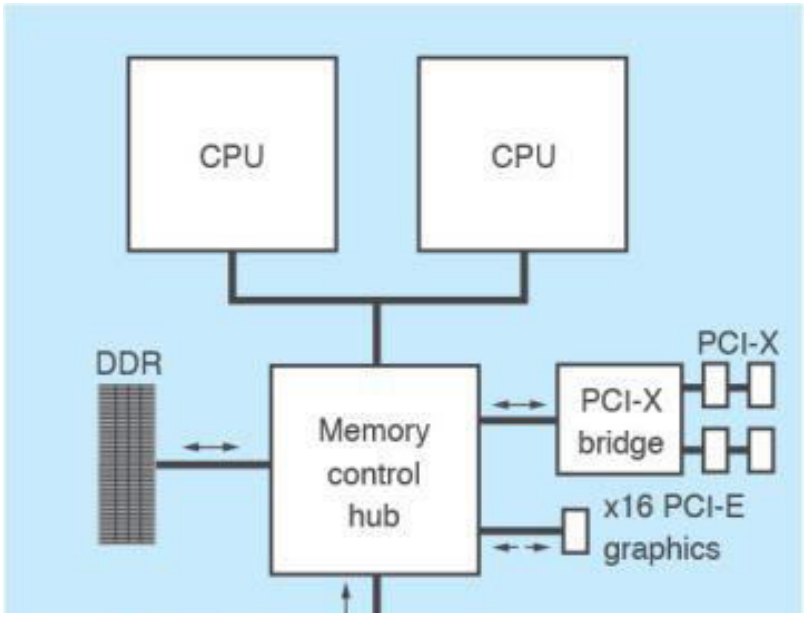
Image credit: "The free Lunch is Over" by Herb Sutter, Dr. Dobbs 2005

Summary: Problems with single "Faster" CPU

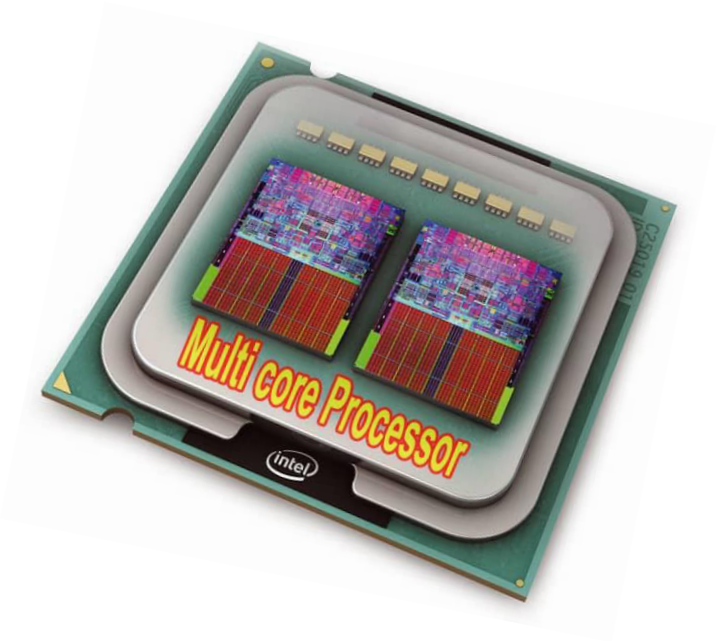
- **Single-core processors** reached their **physical limits**:
  - **Overheating issues**
  - Huge **power consumption requirements**
  - **Difficult to design**



[Picture credits: Intel Corporation,]



Multi-Core Era



[Picture credit: https://i.ibb.co/4ftNR3z/mlcore-copy.jpg

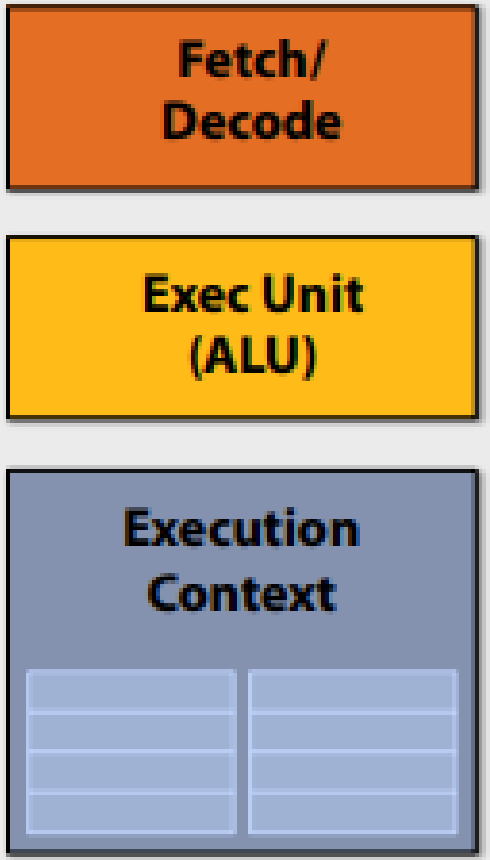
Serial Performance Scaling is Over

- **Cannot** continue to **scale processor** frequencies
  - **no 20 GHz chips**

- **Cannot** continue to **increase power consumption**
  - **can't melt chip**

- **Can continue to increase transistor density**
  - **as per Moore's Law**

Idea of Multicore Era Processor



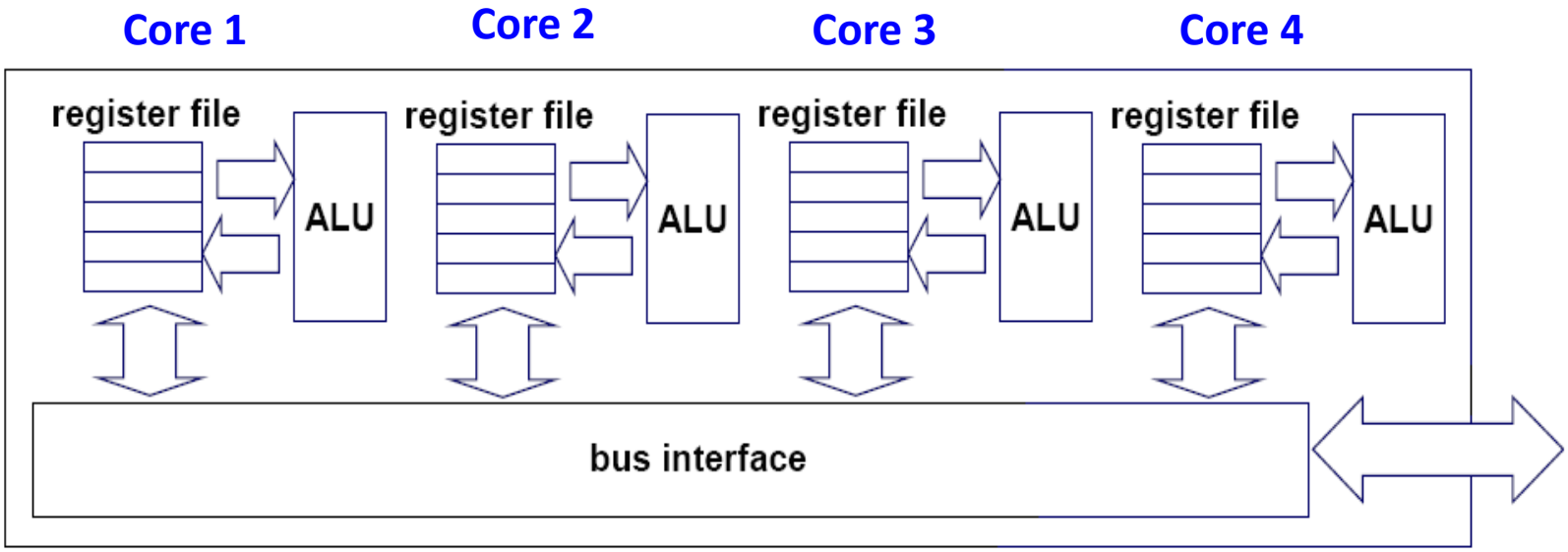
Idea #1:

Rather than use transistors to increase sophistication of processor logic that accelerates a single instruction stream (e.g., out-of-order and speculative operations)

Use increasing transistor count to add more cores to the processor

Multi-core architectures

- Replicated multiple processor on a single die/chip



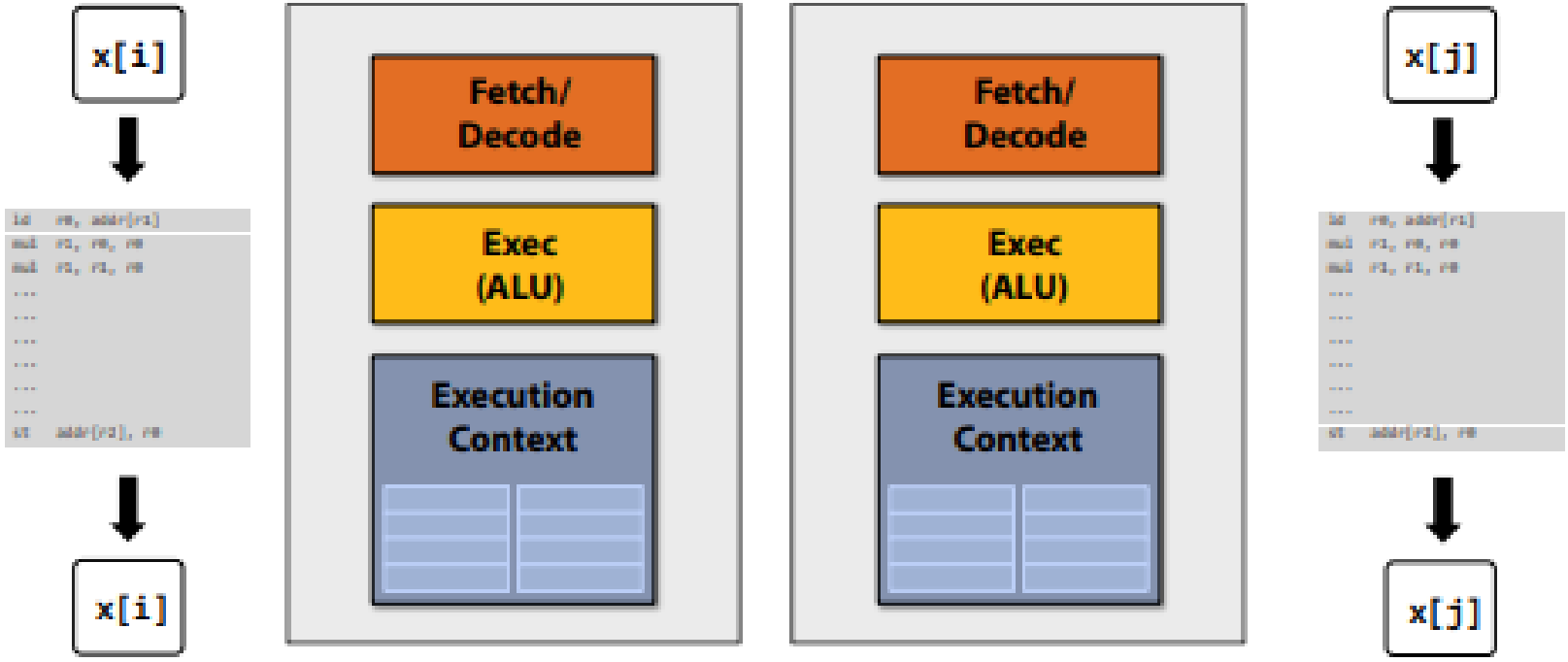
Multi-core CPU chip

The “New” Moore’s Law

- **Computers no longer get faster**, just **wider**
- You *must* re-think your algorithms to be parallel !
- **Data-parallel** computing is **most scalable solution**:

~~2 cores~~ ~~4 cores~~ ~~8 cores~~ **16 cores…**

Idea of Multicore Era Processor



Simpler cores: each core may be slower at running a single instruction stream than our original "fancy" core (e.g., 25% slower)

But there are now two cores: $2 \times 0.75 = 1.5$ (potential for speedup!)

But our program expresses no parallelism

```
void sinx(int N, int terms, float* x, float* y)
{
   for (int i=0; i<N; i++)
   {
      float value = x[i];
      float numer = x[i] * x[i] * x[i];
      int denom = 6;  // 3!
      int sign = -1;

      for (int j=1; j<=terms; j++)
      {
         value += sign * numer / denom;
         numer *= x[i] * x[i];
         denom *= (2*j+2) * (2*j+3);
         sign *= -1;
      }

      y[i] = value;
   }
}
```

This C program will compile to an instruction stream that runs as one thread on one processor core.

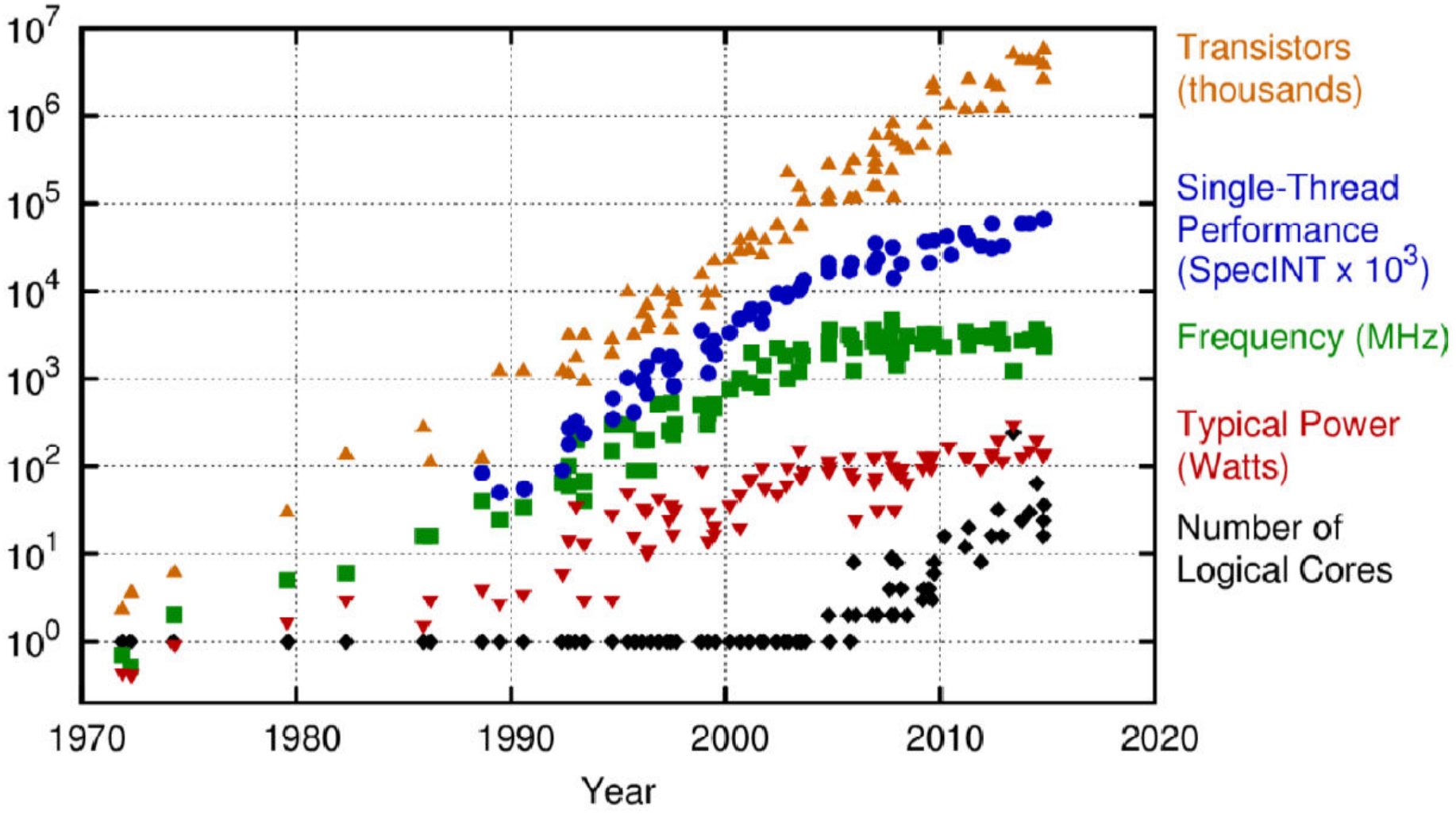
If each of the simpler processor cores was 25% slower than the original single complicated one, our program now runs 25% slower than before.

Example: expressing parallelism using C++ threads

```
typedef struct {
   int N;
   int terms;
   float* x;
   float* y;
} my_args;

void my_thread_func(my_args* args)
{
   sinx(args->N, args->terms, args->x, args->y); // do work
}

void parallel_sinx(int N, int terms, float* x, float* y)
{
   std::thread my_thread;
   my_args args;

   args.N = N/2;
   args.terms = terms;
   args.x = x;
   args.y = y;

   my_thread = std::thread(my_thread_func, &args); // launch thread
   sinx(N - args.N, terms, x + args.N, y + args.N); // do work on main thread
   my_thread.join();  // wait for thread to complete
}
```

```
void sinx(int N, int terms, float* x, float* y)
{
   for (int i=0; i<N; i++)
   {
      float value = x[i];
      float numer = x[i] * x[i] * x[i];
      int denom = 6;  // 3!
      int sign = -1;

      for (int j=1; j<=terms; j++)
      {
         value += sign * numer / denom
         numer *= x[i] * x[i];
         denom *= (2*j+2) * (2*j+3);
         sign *= -1;
      }

      y[i] = value;
   }
}
```

Technology Trends and Multicores



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2015 by K. Rupp

Figure credit: Karl Rupp. https://www.karlrupp.net/wp-content/uploads/2015/06/40-years-processor-trend.png.

Example Parallel Machines

Examples of parallel machines/hardware:

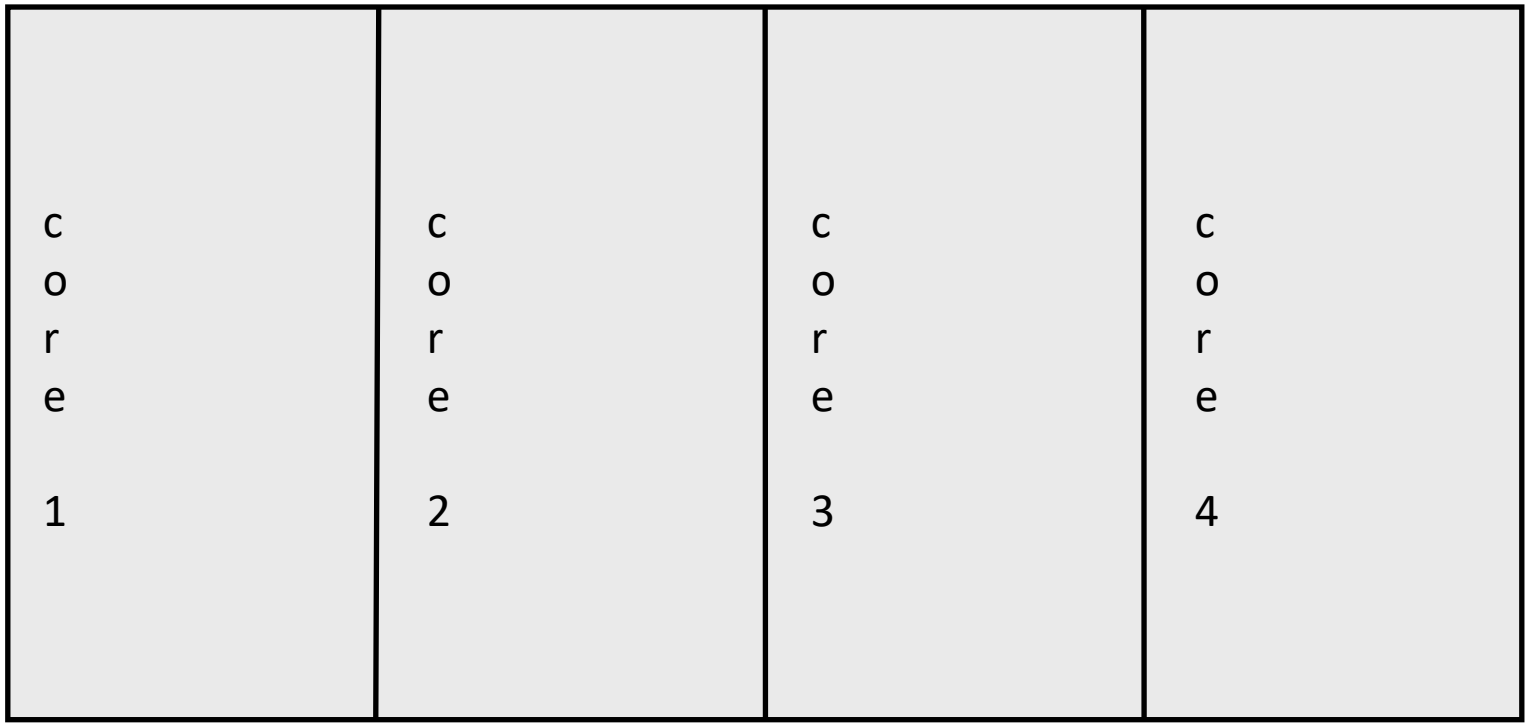
- A **Chip Multi-Processor** (CMP) contains **multiple cores** on a **single chip**

- A **shared memory multi-processor** by connecting **multiple processors** to a **single memory system**

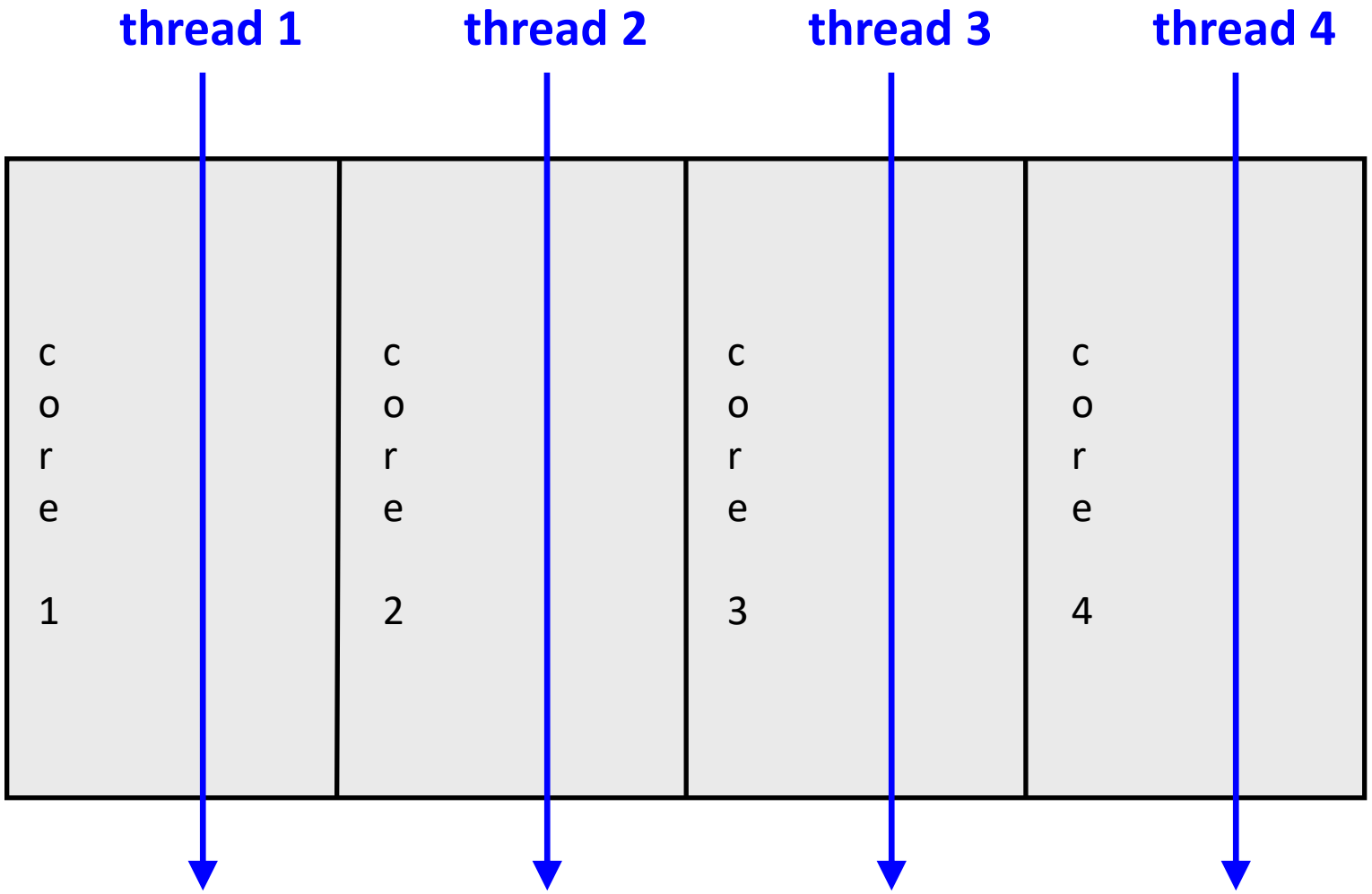
- A **distributed computer** contains **multiple machines combined** together with a **network** such as **Cluster**, **Grid**, **Cloud**, etc.

Multi-core CPU chip

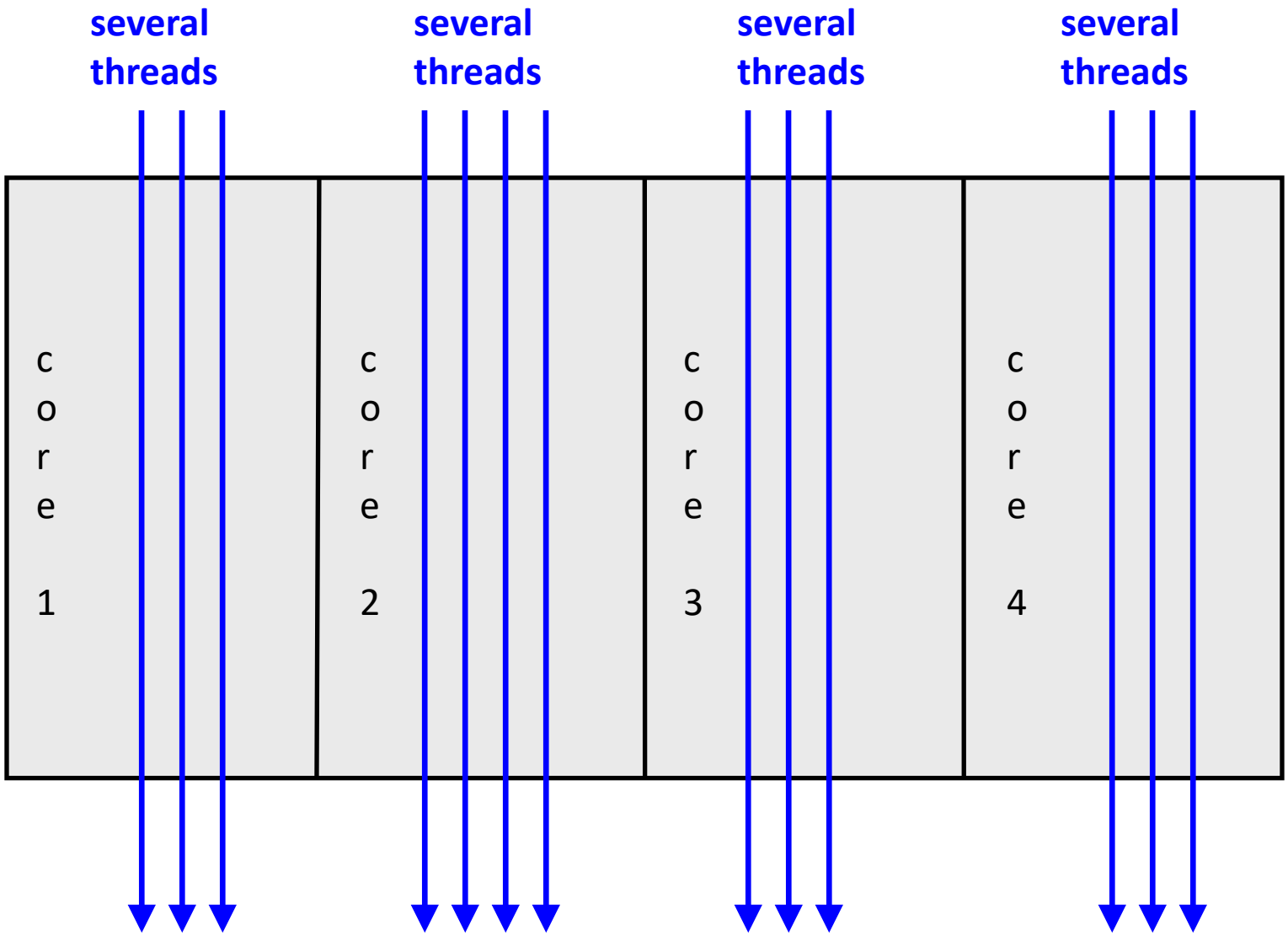
- The **cores** fit on a **single processor socket**
- Also called **Chip Multi-Processor** (**CMP**)



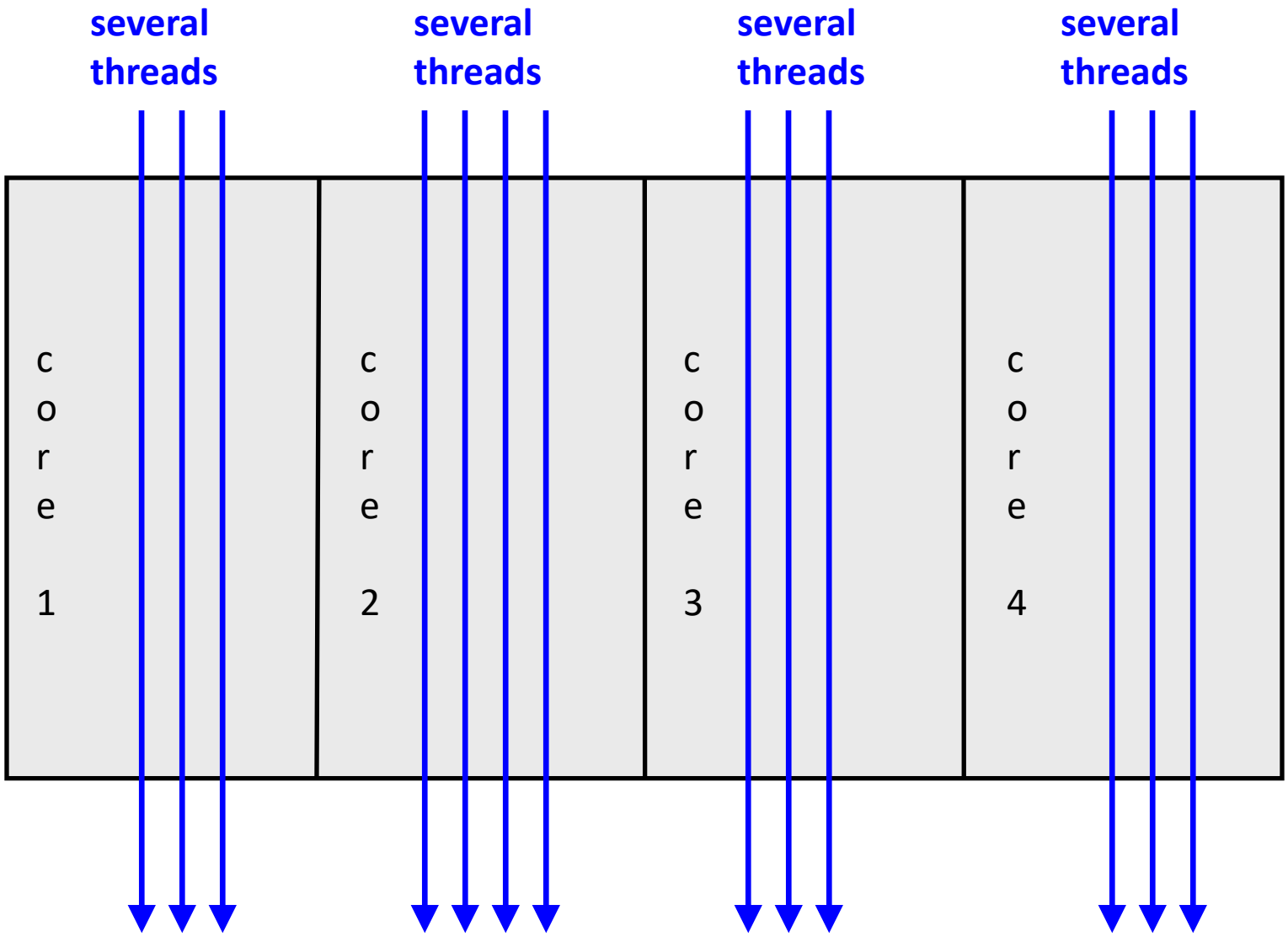
Multi-core CPU chip



Threads may be time-sliced (like a uniprocessor)

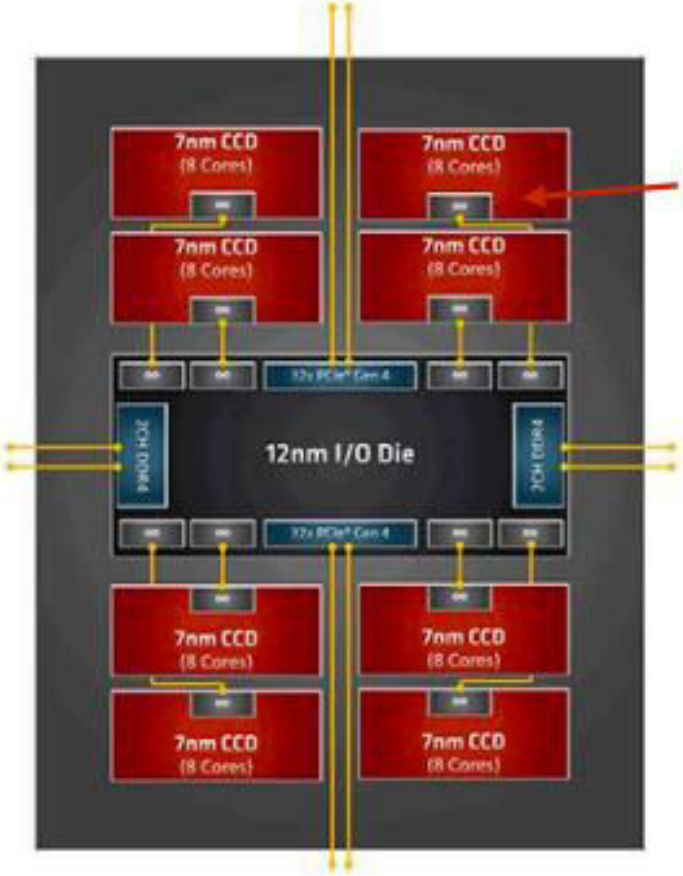


Other Option: Simultaneous Multithreading



AMD Ryzen Threadripper 3990X

64 cores, 4.3 GHz



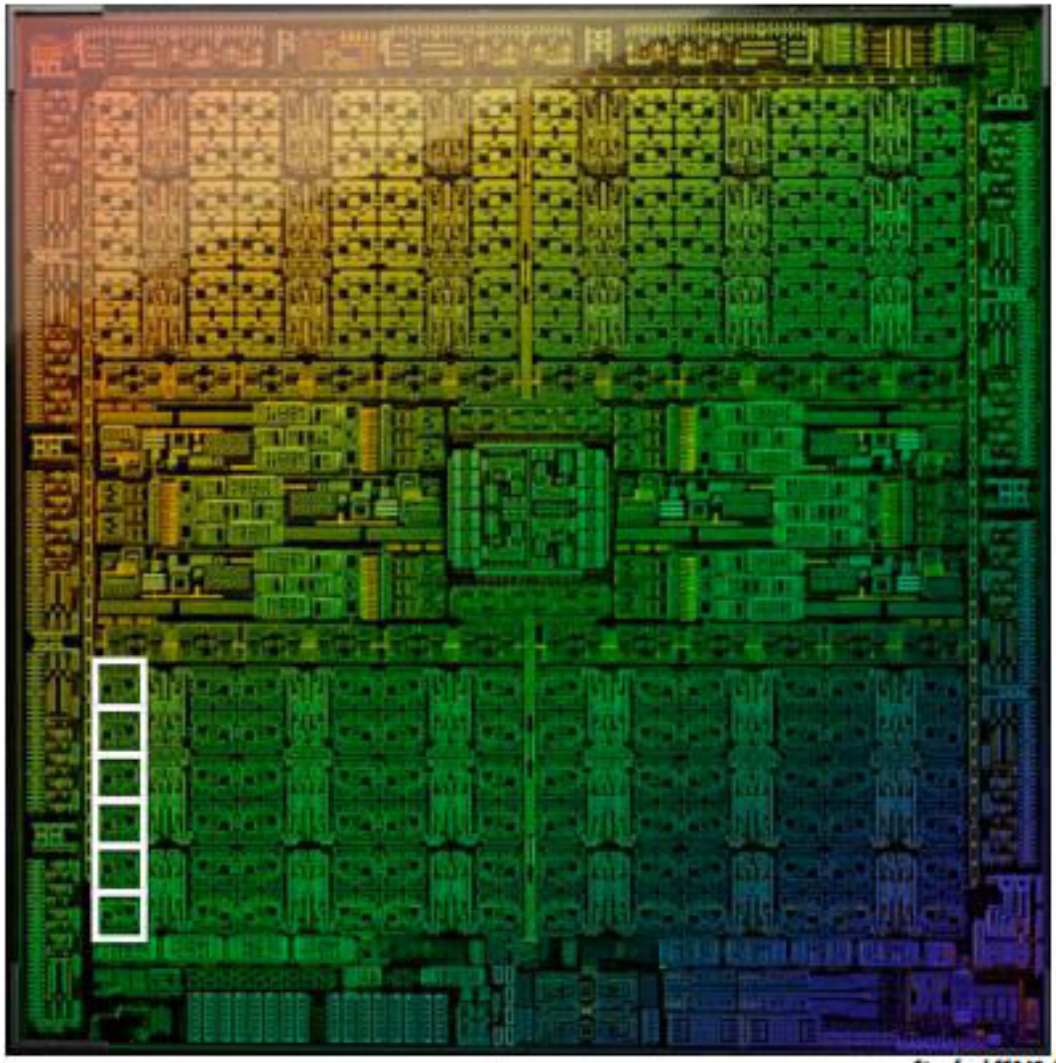
Four 8-core chiplets

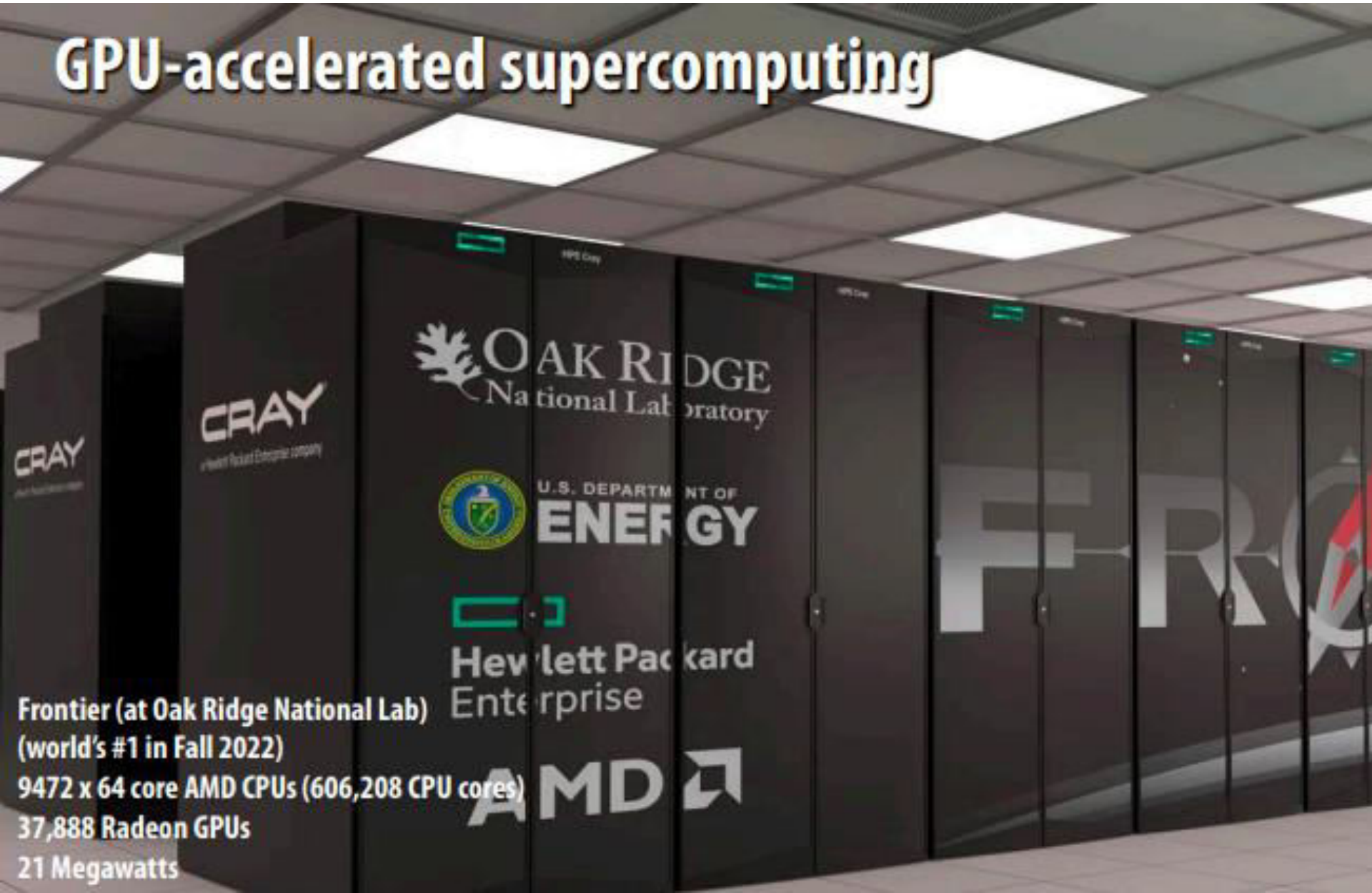
NVIDIA AD102 GPU

GeForce RTX 4090 (2022)

76 billion transistors

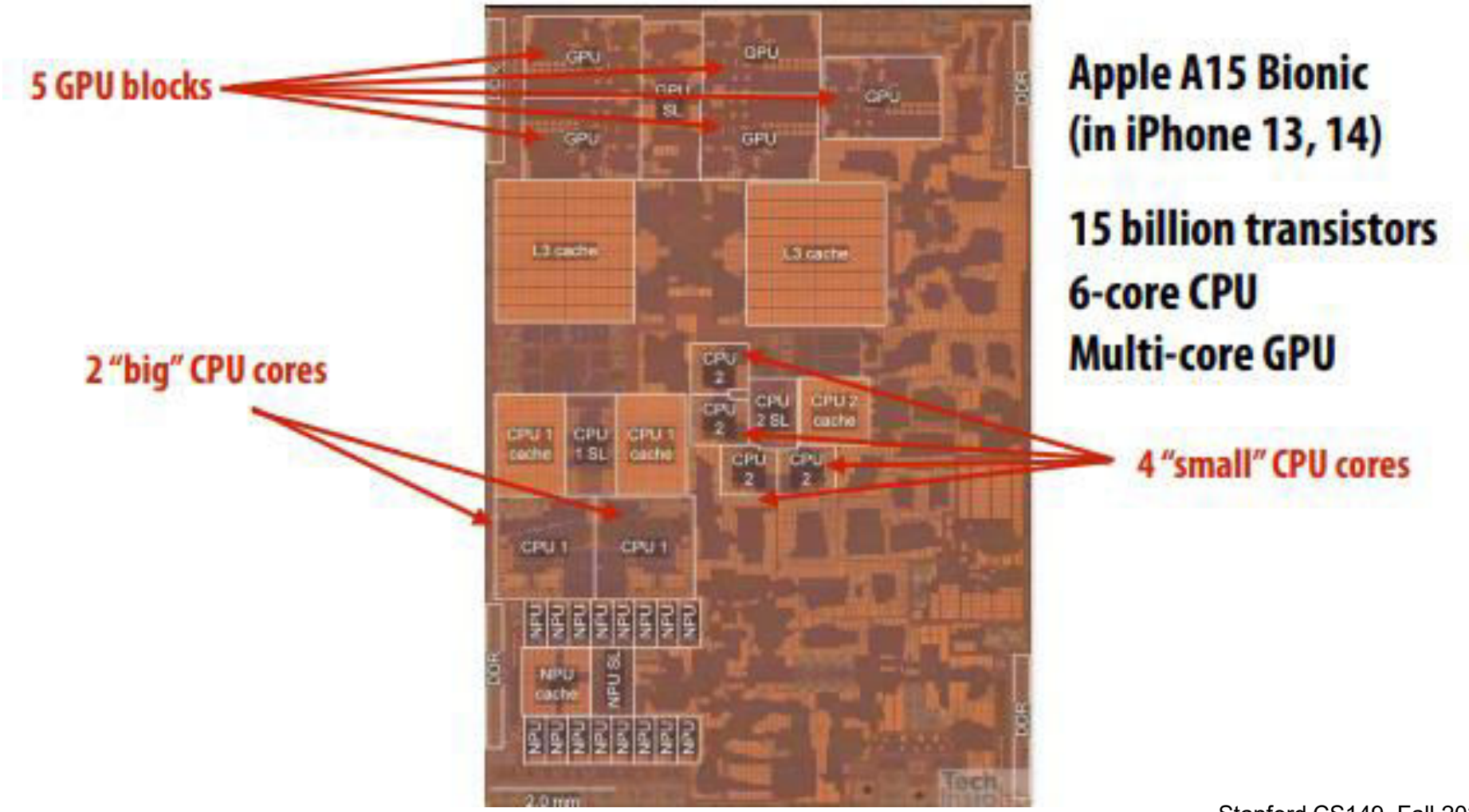
18,432 fp32 multipliers organized in 144 processing blocks (called SMs)



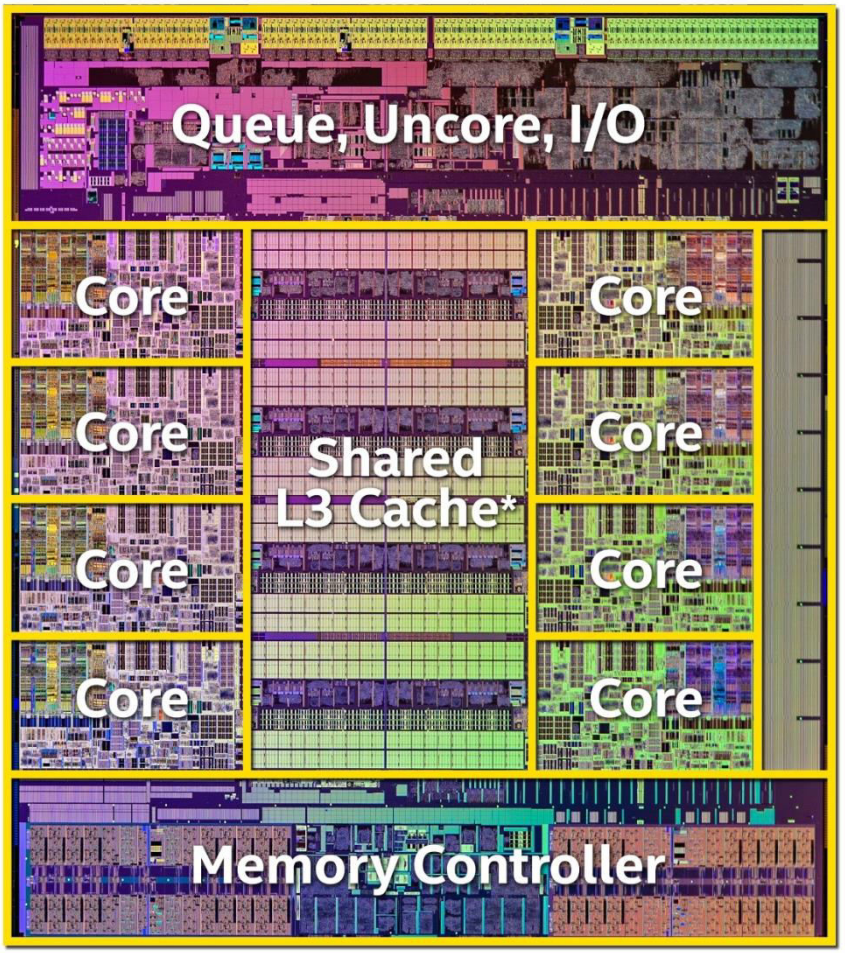

GPU-accelerated supercomputing
CRAY
OAK RIDGE
National Laboratory
U.S. DEPARTMENT OF
ENERGY
Hewlett Packard
Enterprise
AMD
Frontier (at Oak Ridge National Lab)
(world's #1 in Fall 2022)
9472 x 64 core AMD CPUs (606,208 CPU cores)
37,888 Radeon GPUs
21 Megawatts

Mobile parallel processing

Power constraints also heavily influence the design of mobile systems.



8-Core Core-i7 Processor (Extreme Edition)



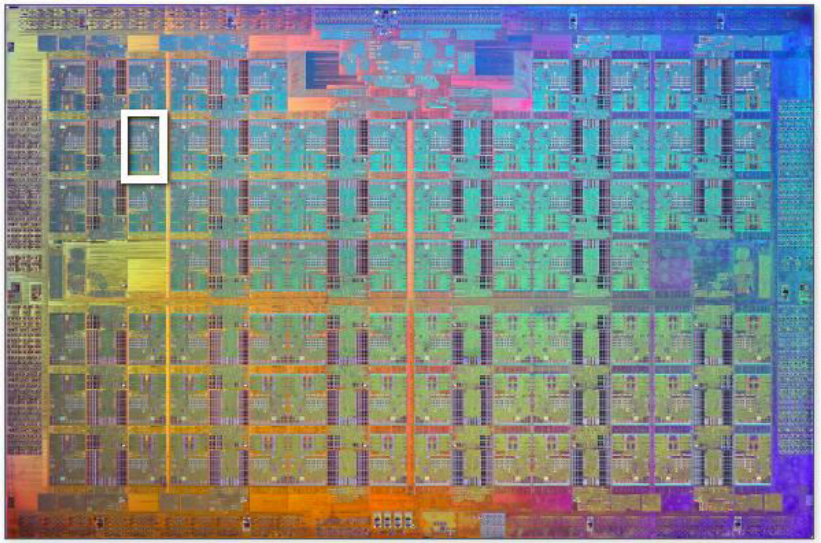
Intel® Core™ i7-5960X Processor Extreme Edition
Transistor count: 2.6 Billion
Die size: 17.6mm x 20.2mm

* 20MB of cache is shared across all 8 cores

http://simplecore.intel.com/newsroom/wp-content/uploads/sites/11/HSW-E-Die-Mapping-Hi-Res.jpg

Intel Xeon Phi 7290 "coprocessor" (2016)

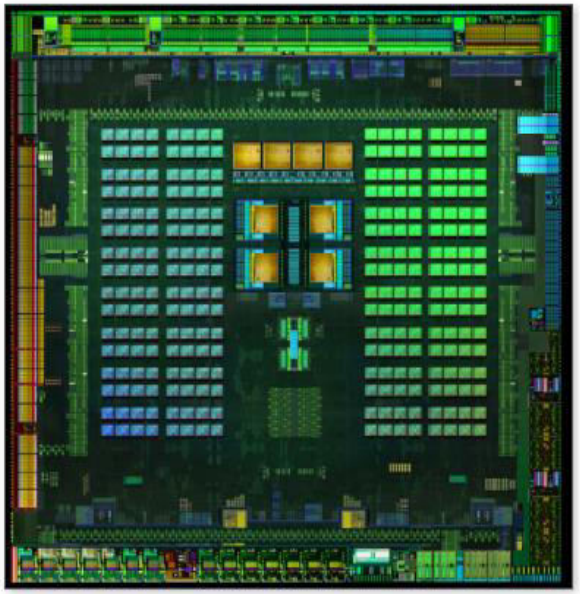
72 cores (1.5 Ghz)



Mobile Parallel Processing

Power constraints heavily influence design of mobile systems

Apple A9: (in iPhone 6s)
Dual-core CPU + GPU + image processor and more on one chip



NVIDIA Tegra X1:
4 ARM A57 CPU cores +
4 ARM A53 CPU cores +
NVIDIA GPU + image processor...

Mobile Parallel Processing

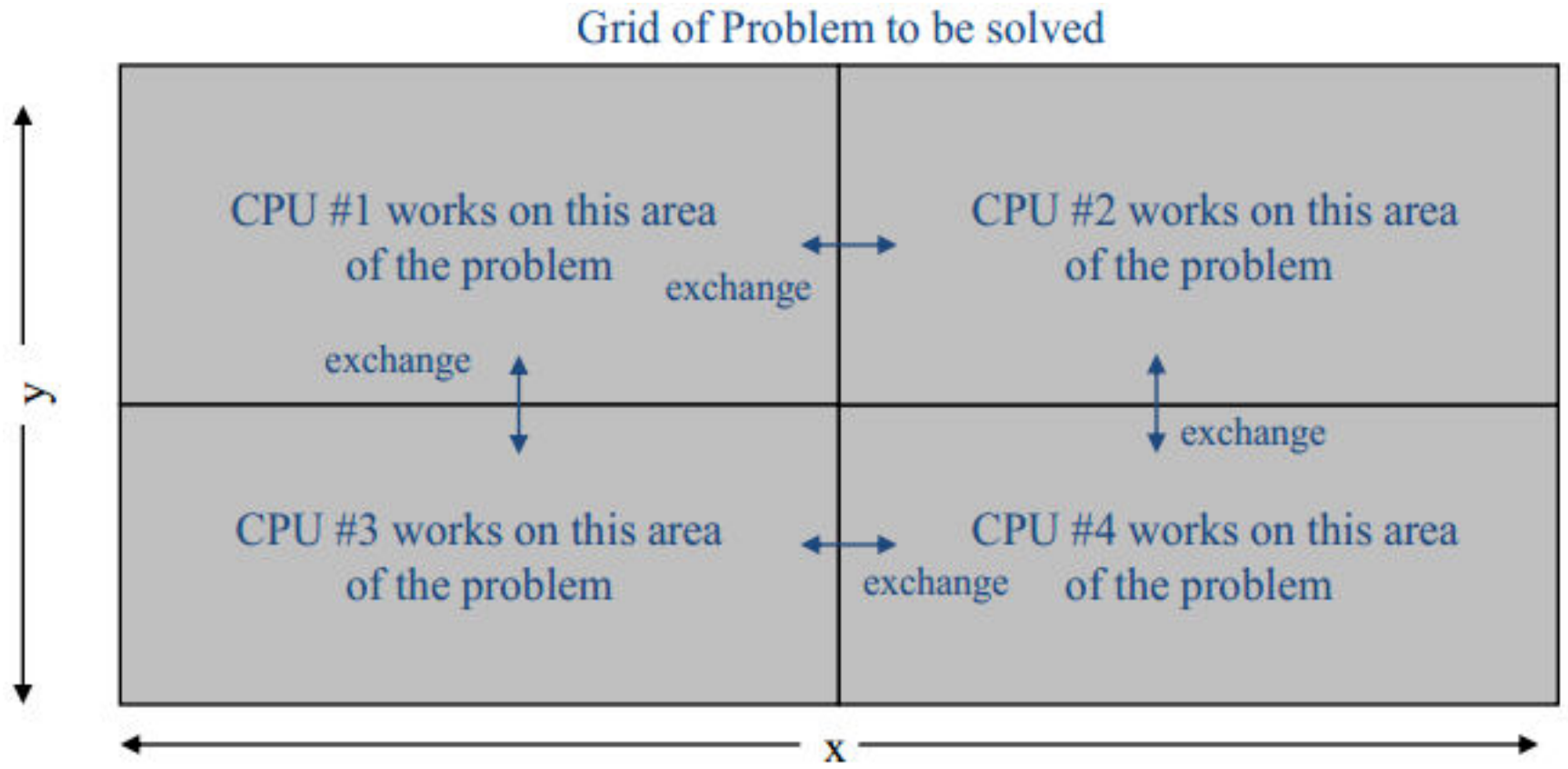
Raspberry Pi 3

Quad-core ARM A53 CPU

Parallel and Distributed Computing?

- **What is Parallel and Distributed Computing?**
  - Use of **multiple processors** or **machines** **working together** on a **common task**
  - Each **processor/machine** works on its **section** of the **problem**
  - **Processors** may **exchange information**

Parallel and Distributed Computing?



Concurrency vs Parallelism

- **Concurrency** is when two or more tasks can start, run, and complete in overlapping time periods. It doesn't necessarily mean they'll ever both be running at the same instant. For example, multitasking on a single-core Machine.

- **Parallelism** is when tasks literally run at the same time, e.g., on a multicore processor.

Parallelism = Concurrency + Hardware Support

Some Terminologies

- **Task**
  - A **logically discrete section** of **computational work**.
  - A task is typically a **program** or **program-like set** of **instructions** that is **executed** by a **processor**.

- **Parallel Task**
  - A **task** that can be **executed** by **multiple processors safely** (***producing correct results***)

- **Serial Execution**
  - **Execution** of a **program sequentially**, one statement at a time using one processor.

Why Parallel and Distributed Computing?

- **Goal: solve large problems** more **quickly**

- **Parallelism provides**:
  - **Solving large problems** in **same time** (or ***even less***)
  - **Solve fixed size problems faster**

Why do parallel computing?

Two main aspects:

1. **Technology Push (What we have discussed so far)**
2. **Applications Pull**

- *Acknowledgements: Department of Computer Science Rice University*

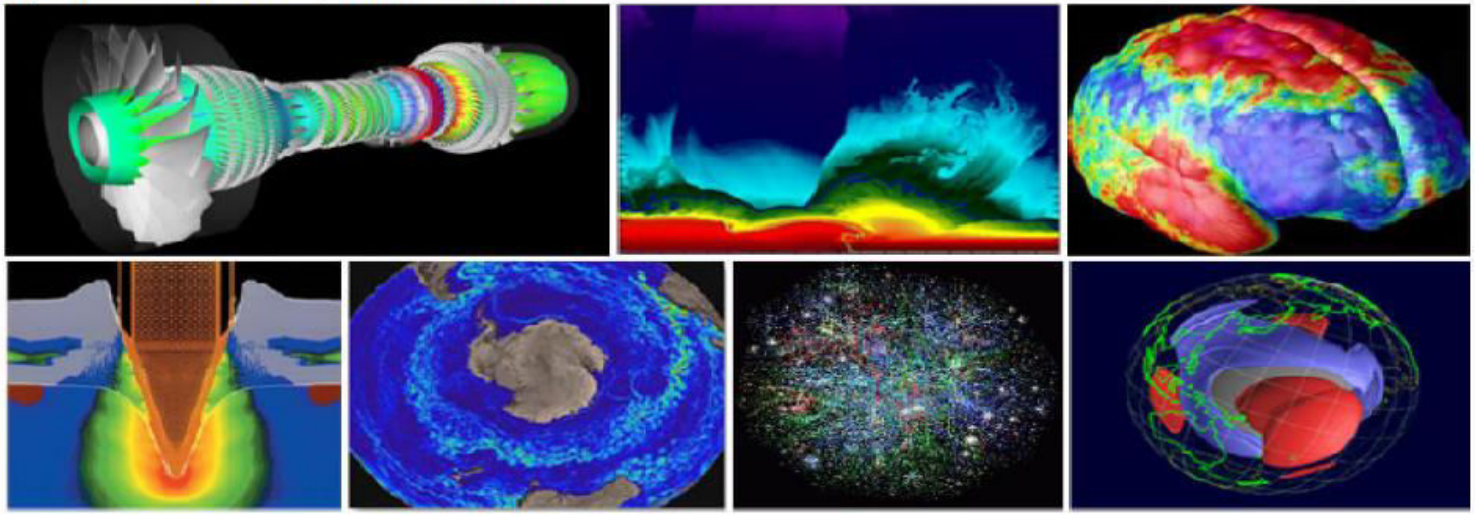
Why do parallel computing?

Application Pull

Why do parallel computing?

Historically, parallel computing has been considered to be "the high end of computing", and has been used to model difficult scientific and engineering problems found in the real world. Some examples:

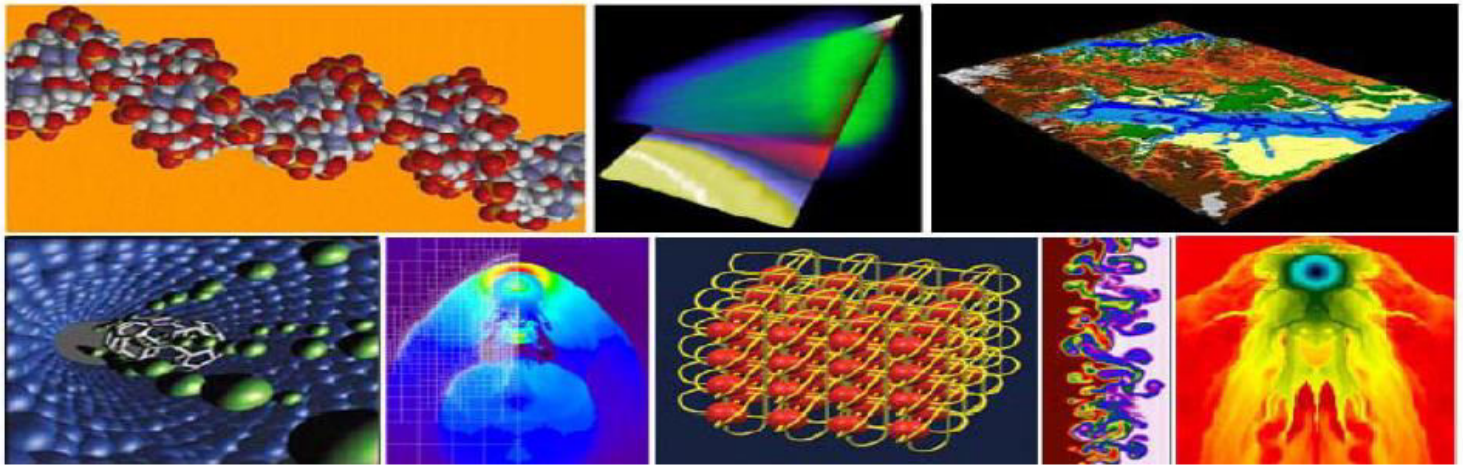
- Atmosphere, Earth, Environment, Space Weather
- **Physics - applied, nuclear, particle, condensed matter, high pressure, fusion, photonics**
- Bioscience, Biotechnology, Genetics
- Chemistry, Molecular Sciences
- Geology, Seismology
- Mechanical and Aerospace Engineering
- Electrical Engineering, Circuit Design, Microelectronics
- Computer Science, Mathematics



Why do parallel computing?

Today, commercial applications provide an equal or greater driving force in the development of faster computers. These applications require the processing of large amounts of data in sophisticated ways. For example:

- Databases, data mining
- Oil exploration
- Web search engines, web based business services
- Medical imaging and diagnosis
- Pharmaceutical design
- Management of national and multi-national corporations
- Financial and economic modeling
- Advanced graphics and virtual reality, particularly in the entertainment industry
- Networked video and multi-media technologies
- Collaborative work environments



Types of Parallelism (Applications)

- Instruction-level parallelism (ILP)
  - Multiple instructions from the same instruction stream can be executed concurrently
  - Generated and managed by hardware (superscalar) or by compiler (VLIW)
  - Limited in practice by data and control dependences

- Thread-level or task-level parallelism (TLP)
  - Multiple threads or instruction sequences from the same application can be executed concurrently
  - Generated by compiler/user and managed by compiler and hardware
  - Limited in practice by communication/synchronization overheads and by algorithm characteristics

Types of Parallelism (Applications)

- Data-level parallelism (DLP)
  - Instructions from a single stream operate concurrently on several data
  - Limited by non-regular data manipulation patterns and by memory bandwidth

- Transaction-level parallelism
  - Multiple threads/processes from different transactions can be executed concurrently
  - Limited by concurrency overheads

Course Topics/Modules

1. Course topics:

   Will be uploaded on GCR.

1. Course Divided Into:

   1. Module-1 [4 weeks]
   2. Module-2 [5 Weeks]
   3. Module-3 [4 Weeks]
   4. Module-4 [2 Weeks]

Detailed Course Contents (1/2)

1. **Module-1 [4 Weeks]**
   - Fundamentals of Parallel and Distributed Computing
   - Need for Parallelism, Parallelizable vs Un-Parallelizable Tasks, Domain and Functional Decomposition
   - Flynns Taxonomy, Parallel Architectures (Shared Nothing, Shared Disk, Shared Memory),
   - Amdahl's Law vs Gustafson's law, Speedup, Efficiency, Measuring Parallelism: Work Done and Speedup
   - Data Parallelism Vs Task Parallelism,
   - Parallelism Granularity, Load balancing, Concurrency and Synchronization
2. **Module-2 [5 Weeks]**
   - Introduction to Parallel Programing Environments
   - Shared Memory Systems: Parallel Algorithms using Open MP - (Mutual Exclusion, Reduction, locks),
   - Distributed Memory System: MPI

Detailed Course Contents (2/2)

3. Module-3: (4 weeks)

- High Performance Computing using GPUs, GPU Architecture and Programming Models (CUDA/OpenCL)
- Accelerating Scientific and ML/DL Applications using Hybrid Architectures

4. Module-4: (2 weeks)

- Performance Modeling and Profiling Techniques
- Dependence Analysis
- Modern Parallel Programming Frameworks (optional)
- Compiler Optimizations (optional)

Grading Policy

Grading policy: **Absolute grading**

Assignments	4	8%
Project	1	8%
Quizzes	5-7	17%
Sessional-I	1	12%
Sessional-II	1	15%
Final Term Exam	1	40%

* Some of the exams may be open-book (Only if announced)

Course Learning Outcomes (CLOs)

1. Demonstrate understanding of various concepts involved in parallel and distributed computer architectures.
2. Implement different parallel and distributed programming paradigms and algorithms using Message-Passing Interface (MPI) and Multithreading frameworks(OpenMP)
3. Perform analytical modeling, dependence, and performance analysis of parallel algorithms and programs.

Course Motivations

- Why study Parallel Computing?
  - Today, serial computers do not exists.
  - Even, your mobile phone is a parallel machine.
  - CPU's Clock frequencies are getting lower.
  - Number of processors are increasing.
  - Application will get slower, if we don't learn the skill to develop parallel programs

Course Aims

- Parallel and distributed computing overview with an introduction to parallel architectures.

- Learning fundamental concepts of parallel program design and dependence analysis.

- Understanding performance aspects of parallel programs and analyzing it (Profiling, Debugging).
- Programming parallel machines using:
  - OpenMP and MPI
  - OpenCL
- If time permitted, Modern Parallel Programming Frameworks (SYCL, OmpSs, etc.), Go/Hadoop/Anyother

Reference Books

- Anshul Gupta, George Karpis, Vipin Kumar, Ananth Grama, Introduction to Parallel Computing, Second Edition, ISBN-10 : 0201648652, Pearson

- Michael J. Quinn, Parallel Programming in C with Mpi and Openmp, 1st Edition, ISBN-10 : 0072822562, McGraw-Hill Science/Engineering/Math

- Aaftab Munshi, Benedict Gaster, Timothy G. Mattson, James Fung,Dan Ginsburg, OpenCL Programming Guide, 1st Edition, Publisher: Addison-Wesley Professional; 1 edition (July 23, 2011)

- Tom White, Hadoop: The Definitive Guide: Storage and Analysis at Internet Scale, 4th Edition, ISBN: 1491901632, O'Reilly Media

- An Introduction to Parallel Programming, 1st edition, Peter Pacheco

- David B. Kirk, Wen-mei W. Hwu, “Programming Massively Parallel Processors A Hands-on Approach”, 3rd Edition (November 24, 2016), Elsevier.

Course Coordination

- **Lecture slides and other material will be shared on Google classroom:**

Link:
https://classroom.google.com/c/ODQwMTk4MjY3MTQ0?cjc=pst24abl

Course Policies

As per course outline.

Any Questions ?

Acknowledgements

- Dr. Muhammad Aleem
- Dr. Arshad Islam
- Dr. Imran Ashraf
- Stanford University
- University of Innsbruck
- ALPEN-ADRIA-UNIVERSITÄT KLAGENFURT
- Rice University
- Carnegie Mellon University
- … several other sources